BASES DE DATOS · 0484 · CFGS DAM / DAW

RA 6 · Diseño de bases de datos

Diseña modelos relacionales normalizados interpretando diagramas entidad/relación. Del enunciado al esquema relacional, con el caso PixelVault.

AUTOR

Alejandro Martínez Cuesta

IES Comercio · Logroño (La Rioja) · Curso 2026/2027

Índice

Inicio 3

El caso PixelVault 6

APARTADOS

1. El proceso de diseño 9

2. El modelo entidad/relación 12

3. Débiles y E/R ampliado 21

4. El modelo relacional 30

5. Del E/R a las tablas 34

6. Reglas de integridad 44

7. Normalización 49

8. Restricciones no representables 54

9. Oracle SQL Data Modeler 59

REPASO

Ejercicios resueltos 65

Autoevaluación 76

Glosario 84

APUNTES · CURSO 2026/2027 · UD 2

Diseño de Bases de Datos

Del enunciado de un cliente al modelo relacional normalizado: diagramas entidad/relación, paso a tablas, integridad y normalización. Todo ello construyendo, paso a paso, la base de datos de una tienda digital de videojuegos.

Resultado de aprendizaje 6 · Bases de Datos (0484) · CFGS DAM / DAW · IES Comercio · Logroño (La Rioja)

Resultado de aprendizaje

RA 6

Diseña modelos relacionales normalizados interpretando diagramas entidad/relación.

Estos son los criterios de evaluación del RA 6 y el apartado donde se trabaja cada uno. Verás las etiquetas **CE 6x** junto a los títulos de las secciones.

Criterio	Qué se evalúa	Dónde
CE 6a	Se han utilizado herramientas gráficas para representar el diseño lógico.	Apartados 2, 3 y 9
CE 6b	Se han identificado las tablas del diseño lógico.	Apartados 4 y 5
CE 6c	Se han identificado los campos que forman parte de las tablas del diseño lógico.	Apartados 4 y 5
CE 6d	Se han analizado las relaciones entre las tablas del diseño lógico.	Apartado 5
CE 6e	Se han identificado los campos clave.	Apartados 4 y 5
CE 6f	Se han aplicado reglas de integridad.	Apartado 6
CE 6g	Se han aplicado reglas de normalización.	Apartado 7
CE 6h	Se han analizado y documentado las restricciones que no pueden plasmarse en el diseño lógico.	Apartado 8

El hilo conductor: PixelVault

A lo largo de toda la unidad diseñamos la base de datos de **PixelVault**, una tienda digital (ficticia) que vende videojuegos para PC y consolas. Cada apartado añade una pieza: primero las entidades básicas, después la jerarquía de juegos de pago y *free-to-play*, luego las tablas, la normalización de un ticket de compra… Al final tendrás el diseño completo de una base de datos realista.

→ Lee primero el enunciado del caso

Contenidos

1. El proceso de diseño

Las tres fases del diseño (conceptual, lógico y físico) y qué produce cada una.

2. El modelo entidad/relación

Entidades, atributos, relaciones y cardinalidades en notación Chen.

3. Débiles y E/R ampliado

Entidades débiles, generalización, especialización y agregación.

4. El modelo relacional

Relaciones, tuplas, dominios, claves y el valor NULL.

5. Del E/R a las tablas

Las reglas de transformación, una por una, aplicadas a PixelVault.

6. Reglas de integridad

Integridad de entidad, referencial y de dominio; qué pasa al borrar.

7. Normalización

Dependencias funcionales y formas normales 1FN, 2FN, 3FN y FNBC.

8. Restricciones no representables

Las reglas de negocio que el diagrama no recoge y cómo documentarlas.

9. Oracle SQL Data Modeler

Práctica guiada: del modelo lógico al script DDL con la herramienta de Oracle.

Repaso

Ejercicios resueltos, autoevaluación y glosario.

Herramientas de la unidad

Herramienta	Para qué la usamos
Papel y lápiz	Los primeros bocetos del diagrama E/R: es lo más rápido para pensar.
draw.io (diagrams.net)	Dibujar diagramas E/R en notación Chen para entregar.
Oracle SQL Data Modeler	Modelo lógico y relacional, y generación del script DDL para Oracle.
Oracle SQL Developer	Ejecutar el script generado en Oracle Database XE (lo veremos a fondo en la UD 5).

Cómo usar estos apuntes

- Usa el **buscador** (arriba a la derecha) para encontrar cualquier término.
- Cambia entre **modo claro y oscuro** con el icono del sol o la luna: los diagramas se adaptan.
- Los diagramas grandes se pueden **desplazar en horizontal** si la pantalla es estrecha.
- Al final tienes **ejercicios resueltos** y una **autoevaluación** con las soluciones plegadas.

UD 2 · HILO CONDUCTOR

El caso PixelVault

PixelVault es una tienda digital de videojuegos (ficticia) que vende códigos de descarga para PC y consolas y ofrece a sus clientes una biblioteca en línea, logros, reseñas y lista de amigos. La empresa nos ha encargado el diseño de su base de datos.

Como en cualquier encargo real, los requisitos no llegan todos a la vez: se recogen en **dos reuniones** con el cliente. Con la primera construiremos el modelo entidad/relación básico (apartado 2); con la segunda lo ampliaremos (apartado 3). Después lo pasaremos a tablas, lo normalizaremos y documentaremos las reglas que el diagrama no puede recoger.

Sobre los datos de ejemplo

Los títulos, estudios, fechas de lanzamiento y clasificaciones PEGI de los juegos son **reales**. Los jugadores, pedidos, precios y logros son **inventados** para los ejemplos.

Reunión 1 · Requisitos iniciales

Esto es lo que nos cuenta la responsable de producto de PixelVault en la primera reunión. Los **términos en negrita** son buenos candidatos a entidad, y lo que aparece detrás de «de cada X se guarda…» suelen ser sus atributos.

1. De cada **jugador** registrado se guarda un identificador interno, su *nick* (que no puede repetirse), el correo electrónico, la fecha de nacimiento y la dirección de facturación: calle, número, código postal, ciudad y país. La edad del jugador no se guarda: se calcula a partir de la fecha de nacimiento.
2. De cada **videojuego** del catálogo se guarda un identificador, el título, la fecha de lanzamiento, la clasificación PEGI (3, 7, 12, 16 o 18) y los idiomas en los que está disponible, que suelen ser varios. En la ficha del juego se muestra su nota media, que se calcula a partir de las reseñas de los jugadores.
3. Cada videojuego lo ha desarrollado **un único estudio** (el estudio principal). De cada **estudio** se guarda un identificador, el nombre y el país. Se pueden dar de alta estudios de los que todavía no se vende ningún juego.
4. Cada videojuego pertenece a **uno o varios géneros** (acción, rol, estrategia, *roguelike*…), y en un género puede haber muchos juegos o ninguno todavía.
5. Cada jugador tiene una **biblioteca** con los juegos que posee. Interesa saber en qué fecha adquirió cada juego.
6. PixelVault vende juegos para varias **plataformas** (PC, PlayStation 5, Xbox Series X|S, Nintendo Switch 2…). De cada plataforma se guarda un identificador, el nombre y el fabricante. Para las estadísticas, la tienda registra cuántas **horas ha jugado cada jugador a cada juego en cada plataforma**.
7. Los jugadores pueden agregar a otros jugadores como **amigos**.
8. Algunos juegos son la **secuela directa** de otro juego del catálogo. A efectos de la tienda, un juego tiene como mucho una secuela directa y es secuela, como mucho, de otro juego.
9. Los jugadores realizan **pedidos**. De cada pedido se guarda un número, la fecha y el jugador que lo ha hecho. Un pedido incluye uno o varios juegos, y de cada juego del pedido interesa el **precio que se pagó**, que puede no coincidir con el precio actual (ofertas, rebajas…).

Reunión 2 · Ampliación de requisitos

Dos semanas después, el cliente vuelve con cambios. Esto es muy habitual y es la razón de que el diseño conceptual se haga *antes* de crear ninguna tabla: cambiar un diagrama es barato; cambiar una base de datos en producción, no.

1. Los estudios también tendrán **cuentas de desarrollador** en la tienda. Jugadores y desarrolladores son **usuarios**: de todo usuario se guarda un identificador, el correo, el nombre, los apellidos y la fecha de registro. De los desarrolladores se guarda además su cargo y el estudio en el que trabajan. Una misma persona puede ser jugador y desarrollador a la vez, y hay usuarios (el personal de soporte) que no son ni una cosa ni la otra.
2. Los juegos se dividen en **juegos de pago**, de los que se guarda el precio actual, y juegos **free-to-play**, de los que se guarda si tienen micropagos. Todo juego es exactamente de uno de los dos tipos. **Solo los juegos de pago se venden en pedidos**; los *free-to-play* se añaden gratis a la biblioteca.
3. Cada juego tiene sus **logros**, numerados dentro del juego (logro 1, logro 2, logro 3…). De cada logro se guarda el nombre, la descripción y los puntos que otorga. Interesa saber qué jugadores han desbloqueado cada logro.
4. Algunos juegos tienen **contenido descargable (DLC)**. Cada DLC tiene un código propio, título, precio y fecha de lanzamiento, y pertenece siempre a un juego. Si un juego se retira del catálogo, sus DLC se retiran con él.
5. Los jugadores pueden escribir **reseñas** de los juegos que tienen en su biblioteca, como máximo una por juego. De cada reseña se guarda un identificador, la nota (de 1 a 10), el texto, la fecha y si el jugador recomienda o no el juego.

Reglas de negocio

Además, el cliente nos da estas reglas. Algunas se podrán reflejar en el diagrama; otras no, y tendremos que documentarlas aparte (apartado 8).

1. Para escribir una reseña hay que haber jugado **al menos 2 horas** a ese juego (sumando todas las plataformas).
2. No se puede vender un juego a un jugador cuya edad sea **inferior a la clasificación PEGI** del juego.
3. Una secuela no puede tener una **fecha de lanzamiento anterior** a la del juego original, y un juego no puede ser secuela de sí mismo.
4. Un jugador no puede ser **amigo de sí mismo**.
5. Un DLC solo se puede comprar si el jugador **tiene el juego base** en su biblioteca.

Datos de ejemplo

Estos son algunos de los juegos del catálogo que usaremos en los ejemplos.

id_juego	titulo	estudio	lanzamiento	PEGI	tipo
1	Grand Theft Auto V	Rockstar North (Reino Unido)	2013-09-17	18	de pago
2	Baldur's Gate 3	Larian Studios (Bélgica)	2023-08-03	18	de pago

id_juego	titulo	estudio	lanzamiento	PEGI	tipo
3	The Legend of Zelda: Tears of the Kingdom	Nintendo EPD (Japón)	2023-05-12	12	de pago
4	Elden Ring	FromSoftware (Japón)	2022-02-25	16	de pago
5	Hollow Knight	Team Cherry (Australia)	2017-02-24	7	de pago
6	Hades	Supergiant Games (EE. UU.)	2020-09-17	12	de pago
7	The Witcher 3: Wild Hunt	CD Projekt Red (Polonia)	2015-05-19	18	de pago
8	Stardew Valley	ConcernedApe (EE. UU.)	2016-02-26	7	de pago
9	Fortnite	Epic Games (EE. UU.)	2017-07-25	12	free-to-play
10	Counter-Strike 2	Valve (EE. UU.)	2023-09-27	18	free-to-play

Y estos son algunos jugadores (inventados):

id_jugador	nick	email	fecha_nacimiento	ciudad	pais
101	NoctisRJ	noctis@ejemplo.es	2006-03-14	Logroño	España
102	LaTarasca	tarasca@ejemplo.es	1999-11-02	Haro	España
103	Kirbyto	kirbyto@ejemplo.es	2012-06-30	Calahorra	España
104	Ezio_Lgr	ezio@ejemplo.es	2001-01-21	Logroño	España

Antes de seguir

Relee los requisitos con un lápiz en la mano y subraya los sustantivos que creas que serán **entidades** y los que serán **atributos**. En el apartado 2 comprobarás si has acertado.

UD 2 · APARTADO 1

El proceso de diseño de una base de datos

Antes de escribir un solo `CREATE TABLE` hay que decidir **qué** datos se van a guardar y **cómo** se organizan. Ese trabajo es el diseño de la base de datos, y de él depende que la aplicación funcione bien durante años o que se convierta en una fuente continua de errores. En este apartado vemos las fases del diseño y qué se obtiene en cada una.

1.1 ¿Por qué hace falta diseñar?

Imagina que PixelVault empieza a funcionar guardando sus ventas en una única hoja de cálculo como esta:

num_pedido	fecha	nick	email	titulo	estudio	pais_estudio	precio
5001	2026-09-01	NoctisRJ	noctis@ejemplo.es	Hades	Supergiant Games	EE. UU.	24,50
5001	2026-09-01	NoctisRJ	noctis@ejemplo.es	Hollow Knight	Team Cherry	Australia	14,79
5002	2026-09-02	LaTarasca	tarasca@ejemplo.es	Hades	Supergiant Games	EE. UU.	19,99
5003	2026-09-02	NoctisRJ	noctis@nuevo.es	Elden Ring	FromSoftware	Japón	59,99

Funciona… hasta que deja de hacerlo. Fíjate en los problemas:

- **Redundancia.** El correo de NoctisRJ o el país de Supergiant Games se repiten en muchas filas. Ocupa espacio y, sobre todo, abre la puerta a incoherencias.
- **Anomalía de modificación.** NoctisRJ cambió su correo y solo se actualizó en una fila: ahora la hoja dice dos cosas distintas sobre el mismo jugador.
- **Anomalía de inserción.** No podemos dar de alta un estudio nuevo hasta que se venda alguno de sus juegos, porque cada fila es una venta.
- **Anomalía de borrado.** Si se anula el pedido 5003, perdemos el único registro de que existe FromSoftware y de que es japonés.

Un buen diseño evita estos problemas desde el principio. En el apartado 7 volveremos a esta misma tabla y la arreglaremos con la **normalización**.

1.2 Las fases del diseño

El diseño de una base de datos se hace en fases, de lo más abstracto a lo más concreto. Cada fase toma como entrada el resultado de la anterior.



Fase	Pregunta que responde	Resultado	¿Depende del SGBD?
Análisis de requisitos	¿Qué necesita guardar y consultar el cliente?	Documento de requisitos (como el del caso PixelVault)	No
Diseño conceptual	¿Qué «cosas» hay y cómo se relacionan?	**Diagrama entidad/relación**	No
Diseño lógico	¿Qué tablas, columnas y claves habrá?	**Esquema relacional** normalizado	Solo del tipo de modelo (relacional)
Diseño físico	¿Cómo se implementa en este gestor concreto?	Script **DDL** (tipos de dato, índices, espacio…)	Sí, de Oracle

Análisis de requisitos

Consiste en entrevistarse con el cliente y los usuarios, revisar los documentos que ya usan (tickets, fichas, hojas de cálculo) y dejar por escrito qué datos hay que guardar y qué reglas deben cumplir. Es la fase en la que más errores se cometen, porque el cliente y el informático no hablan el mismo idioma. Por eso se redacta en lenguaje natural y se valida con el cliente.

Diseño conceptual

Se representa la realidad con un **modelo de datos conceptual**, que en este módulo es el **modelo entidad/relación (E/R)** propuesto por Peter Chen en 1976 y ampliado después. El diagrama E/R es independiente de la tecnología: el mismo diagrama sirve para Oracle, MySQL o PostgreSQL. Lo trabajamos en los apartados 2 y 3.

Diseño lógico

El diagrama E/R se transforma en un **esquema relacional**: tablas, columnas, claves primarias y claves ajenas. Después se comprueba que el esquema está **normalizado**. Es el núcleo del RA 6 y ocupa los apartados 4 a 8.

Diseño físico

Se decide cómo se implementa el esquema en un gestor concreto: tipos de dato de Oracle (`NUMBER` , `VARCHAR2` , `DATE` ...), índices, restricciones `CHECK` , espacio de almacenamiento... El resultado es un script en lenguaje de definición de datos (DDL). Lo haremos a fondo en la **UD 5**, y en el apartado 9 verás cómo Oracle SQL Data Modeler genera ese script automáticamente.

> **El diseño es iterativo**
>
> Aunque las fases parecen una línea recta, en la práctica se vuelve atrás a menudo. En PixelVault lo veremos claramente: en la segunda reunión el cliente amplía los requisitos y tenemos que retocar el diagrama E/R antes de seguir.

1.3 Modelos de datos

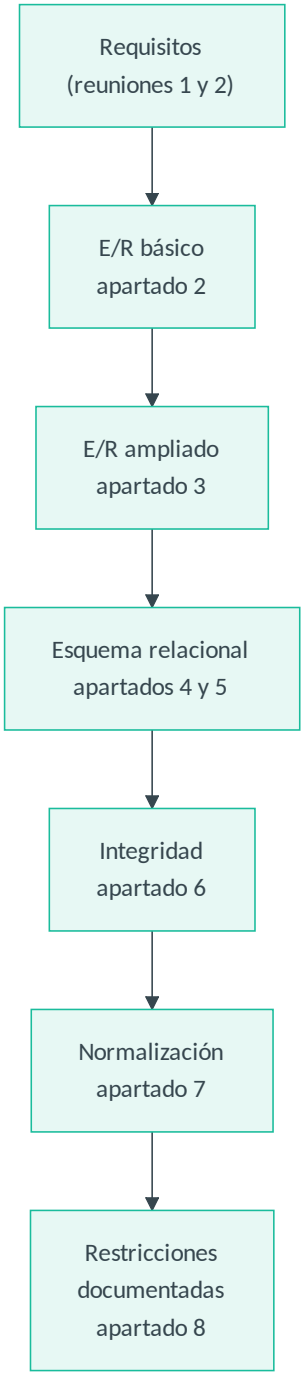
Un **modelo de datos** es un conjunto de conceptos y reglas para describir la estructura de una base de datos. En cada fase se usa un modelo distinto:

Nivel	Modelo	Elementos con los que trabaja
Conceptual	Entidad/relación (E/R)	Entidades, atributos, relaciones, cardinalidades
Lógico	Relacional	Relaciones (tablas), atributos (columnas), tuplas (filas), claves
Físico	El del SGBD (Oracle)	Tablas, tipos de dato, índices, *tablespaces*, restricciones

> **Otros modelos lógicos**
>
> El relacional no es el único modelo lógico: existen el jerárquico, el de red y los modelos NoSQL (documentales, clave-valor, grafos...). Los NoSQL los estudiaremos en la **UD 7**.

1.4 PixelVault en la hoja de ruta

Así avanzará nuestro caso a lo largo de la unidad:



Resumen

- Un mal diseño provoca **redundancia** y **anomalías** de inserción, modificación y borrado.
- El diseño se hace en fases: **requisitos** → **conceptual (E/R)** → **lógico (relacional)** → **físico (Oracle)**.
- El diseño conceptual y el lógico no dependen del gestor concreto; el físico, sí.
- El proceso es **iterativo**: los cambios de requisitos obligan a revisar el diseño, y es mucho más barato hacerlo en el papel que en una base de datos en producción.

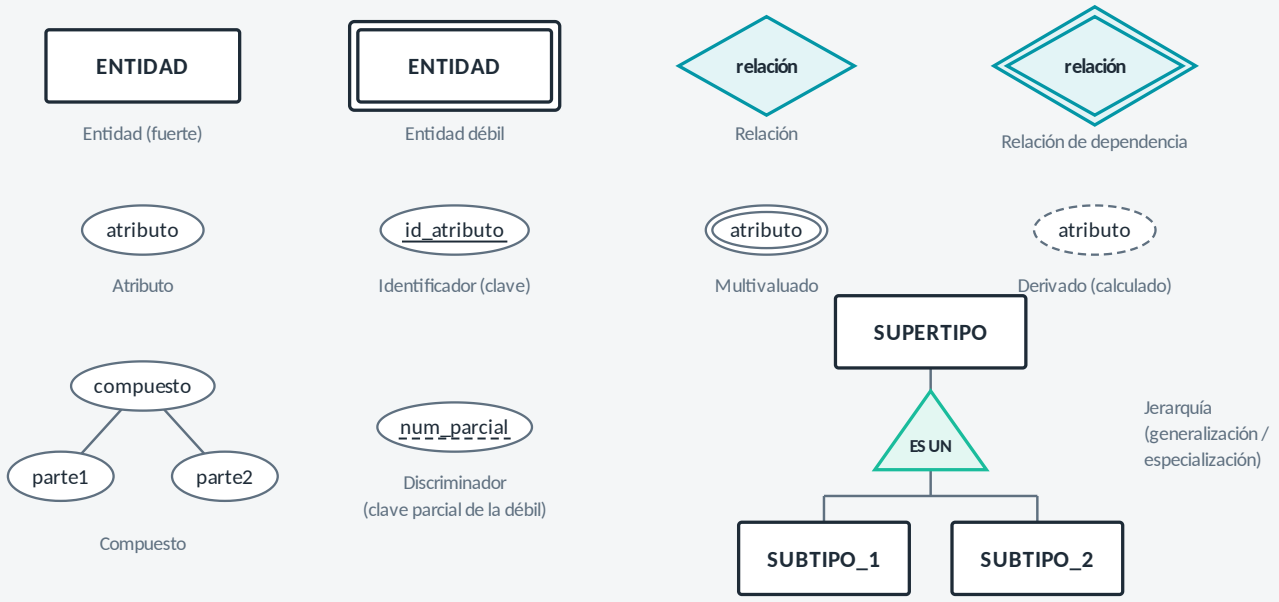
UD 2 · APARTADO 2

El modelo entidad/relación

El modelo entidad/relación (E/R) es la herramienta con la que se hace el **diseño conceptual**: describe qué «cosas» interesan (entidades), qué datos tenemos de ellas (atributos) y cómo se vinculan entre sí (relaciones). En este apartado aprenderás sus elementos básicos y construirás el primer diagrama de PixelVault a partir de la reunión 1 con el cliente.

2.1 La notación que usamos CE 6a

Existen varias notaciones para dibujar diagramas E/R. En este módulo usamos la **notación Chen ampliada**, la habitual en los libros de texto en español. Estos son sus símbolos; los irás viendo uno a uno en este apartado y en el siguiente.



Símbolos de la notación Chen ampliada.

Otras notaciones

Las herramientas profesionales, como Oracle SQL Data Modeler, suelen usar la notación **pata de gallo** (*crow's foot*, también llamada de Ingeniería de la Información) o la de **Barker**. Representan los mismos conceptos con otros símbolos. Las verás en el apartado 9.

2.2 Entidades

Una **entidad** es cualquier objeto, persona, concepto o suceso del mundo real del que interesa guardar información y que se puede distinguir de los demás. Se dibuja con un **rectángulo** y su nombre va en singular y en mayúsculas.

Hay que distinguir entre el **tipo de entidad** (el concepto general: JUGADOR) y cada una de sus **ocurrencias** o ejemplares (el jugador NoctisRJ, el jugador LaTarasca…). En el diagrama se dibujan tipos, no ocurrencias.

En la reunión 1 de PixelVault aparecen estas entidades: **JUGADOR**, **VIDEOJUEGO**, **ESTUDIO**, **GÉNERO**, **PLATAFORMA** y **PEDIDO**.

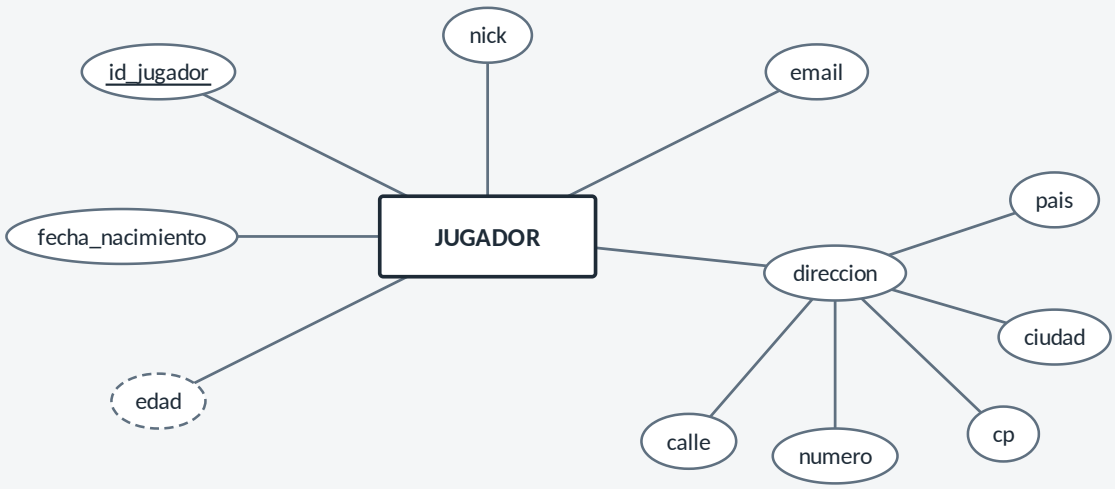
> **¿Entidad o atributo?**
>
> Pregúntate si el concepto tiene **datos propios** y **vida independiente**. El país de un estudio es solo un dato del estudio: es un atributo. En cambio, un género tiene sentido por sí mismo (podemos dar de alta «Metroidvania» aunque aún no haya juegos) y se relaciona con muchos juegos: es una entidad.

2.3 Atributos

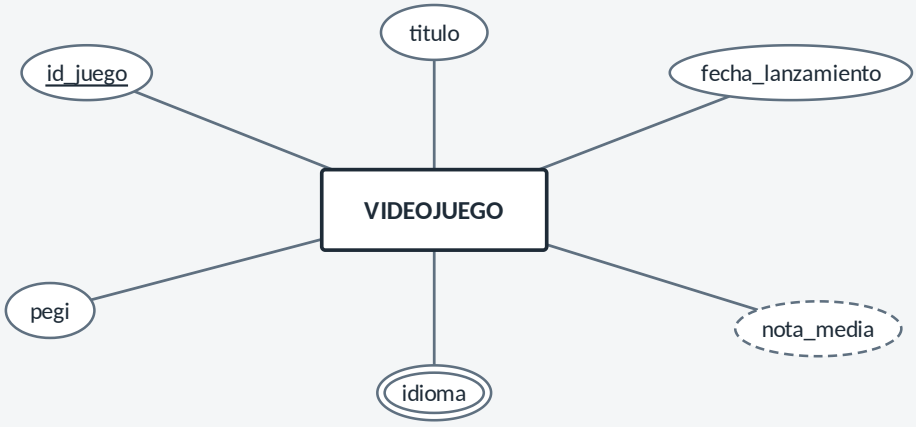
Un **atributo** es una propiedad o característica de una entidad (o de una relación). Se dibuja con una **elipse** unida a su entidad. Cada atributo toma sus valores de un **dominio**, el conjunto de valores válidos: por ejemplo, el dominio de `pegi` es {3, 7, 12, 16, 18}.

Tipos de atributo

Tipo	Qué es	Cómo se dibuja	Ejemplo en PixelVault
Simple	No se puede descomponer	Elipse	`nick`, `titulo`
Compuesto	Formado por otros atributos	Elipse con sus componentes colgando	`direccion` (calle, numero, cp, ciudad, pais)
Multivaluado	Puede tener varios valores para la misma ocurrencia	Elipse doble	`idioma` de un videojuego
Derivado	Se calcula a partir de otros datos	Elipse discontinua	`edad` (de la fecha de nacimiento), `nota_media`
Identificador	Distingue cada ocurrencia de las demás	Nombre subrayado	`id_jugador`, `id_juego`



La entidad JUGADOR: identificador subrayado, atributo compuesto (direccion) y atributo derivado (edad).



La entidad VIDEOJUEGO: idioma es multivaluado (un juego está en varios idiomas) y nota_media es derivado.

Identificadores y claves

Un **identificador** (o clave) es un atributo, o conjunto de atributos, cuyo valor no se repite en dos ocurrencias distintas.

Clave candidata

> Cualquier atributo o conjunto mínimo de atributos que identifica cada ocurrencia. En JUGADOR hay tres: `id_jugador`, `nick` y `email`.

Clave primaria

> La clave candidata que elegimos como identificador principal. Se subraya. En JUGADOR elegimos `id_jugador`.

Clave alternativa

Las claves candidatas que no se han elegido como primaria (`nick` y `email`). Siguen sin poder repetirse; en Oracle se implementarán con `UNIQUE` .

¿Por qué un identificador inventado?

Podríamos usar `nick` como clave primaria, pero los jugadores pueden cambiar de *nick* y la clave primaria no debería cambiar nunca, porque otras tablas la referencian. Por eso se suele usar un identificador interno (`id_jugador`) que no tiene significado para el usuario. Se llama **clave subrogada** o artificial.

Error frecuente

Dibujar `edad` como atributo normal. Si se guarda la edad, dentro de un año será falsa; lo correcto es guardar la **fecha de nacimiento** y calcular la edad cuando se necesite. Lo mismo ocurre con `nota_media` o con el importe total de un pedido.

2.4 Relaciones

Una **relación** es una asociación entre entidades. Se dibuja con un **rombo** unido a las entidades que asocia, y su nombre suele ser un verbo: *desarrolla*, *posee*, *pertenece*…

El **grado** de una relación es el número de entidades que participan en ella:

Grado	Nombre	Ejemplo en PixelVault
1	Reflexiva (unaria o recursiva)	JUGADOR *es_amigo_de* JUGADOR
2	Binaria	ESTUDIO *desarrolla* VIDEOJUEGO
3	Ternaria	JUGADOR *juega* VIDEOJUEGO en PLATAFORMA

La inmensa mayoría de las relaciones son binarias.

2.5 Cardinalidades

La **cardinalidad** indica con cuántas ocurrencias de una entidad se puede relacionar cada ocurrencia de la otra. Es la parte más importante (y la que más fallos provoca) del diagrama, porque de ella dependerá después dónde van las claves ajenas y si admiten nulos.

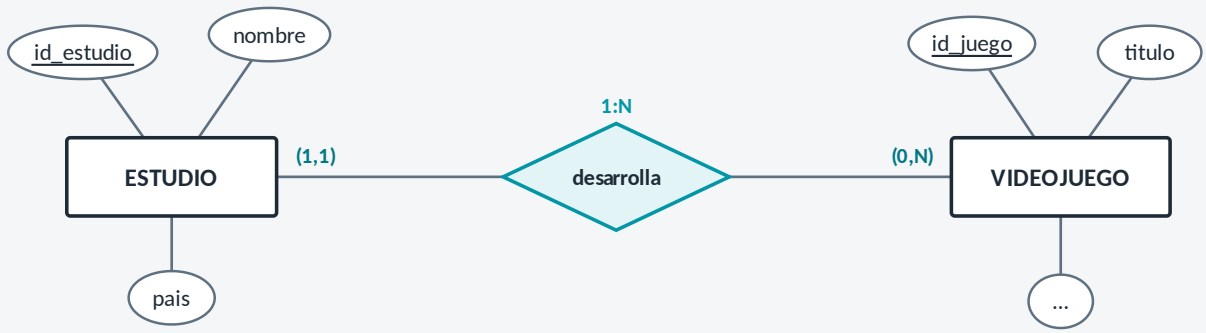
El par (mín, máx) en el extremo opuesto

La cardinalidad de una relación binaria se expresa con dos pares **(mín, máx)**, uno en cada extremo. Cada par se escribe **junto a la entidad que se cuenta**, es decir, en el **extremo opuesto** a la entidad desde la que se hace la pregunta:

> Junto a la entidad **B** se escribe cuántas ocurrencias de **B** (como mínimo y como máximo) se relacionan con **una** ocurrencia de **A**.

- El **mínimo** es 0 (relación **opcional**) o 1 (relación **obligatoria**).

- El **máximo** es 1 o N (muchos).



Requisito 3: cada videojuego lo desarrolla un único estudio, y puede haber estudios sin juegos todavía.

Se lee así, siempre partiendo de **una** ocurrencia del otro lado y leyendo hacia el par:

- Un **estudio** desarrolla **como mínimo 0 y como máximo N** videojuegos: por eso `(0,N)` está junto a **VIDEOJUEGO**. El mínimo 0 indica que puede haber estudios dados de alta sin juegos.
- Un **videojuego** es desarrollado **como mínimo por 1 y como máximo por 1** estudio: por eso `(1,1)` está junto a **ESTUDIO**. Es decir, exactamente uno.

Método de las dos preguntas

Para cada relación binaria A–B, hazte dos preguntas en voz alta y escribe cada respuesta **al otro lado**, junto a la entidad por la que preguntas:

1. «Un **estudio** concreto, ¿cuántos **videojuegos** puede desarrollar?» → de 0 a N → `(0,N)` junto a VIDEOJUEGO.
2. «Un **videojuego** concreto, ¿cuántos **estudios** lo desarrollan?» → de 1 a 1 → `(1,1)` junto a ESTUDIO.

Truco para comprobarlo: lee el diagrama como una frase, de izquierda a derecha y de derecha a izquierda: «un ESTUDIO *desarrolla* de 0 a N VIDEOJUEGOS»; «un VIDEOJUEGO es *desarrollado* por 1 y solo 1 ESTUDIO».

Tipo de correspondencia

A partir de los **máximos** se obtiene el tipo de la relación, que se escribe encima del rombo. Con este convenio, los máximos aparecen **en el mismo lado** en el que se escriben en el tipo:

Tipo	Significado	Ejemplo
1:1	Cada A con una B como máximo, y cada B con una A como máximo	VIDEOJUEGO *es_secuela_de* VIDEOJUEGO
1:N	Cada A con muchas B, pero cada B con una A como máximo	ESTUDIO *desarrolla* VIDEOJUEGO
N:M	Cada A con muchas B, y cada B con muchas A	JUGADOR *posee* VIDEOJUEGO

En *desarrolla*, el máximo junto a ESTUDIO es **1** y el máximo junto a VIDEOJUEGO es **N**: la relación es **1:N** («un estudio, muchos juegos»). Se lee directamente en el diagrama.

Error frecuente

Escribir solo `1` o `N` sin el mínimo. El mínimo parece un detalle, pero no lo es: en el apartado 5 verás que, en una 1:N, el mínimo que hay en el lado 1 decide si la clave ajena es `NOT NULL` (mínimo 1) o admite nulos (mínimo 0).

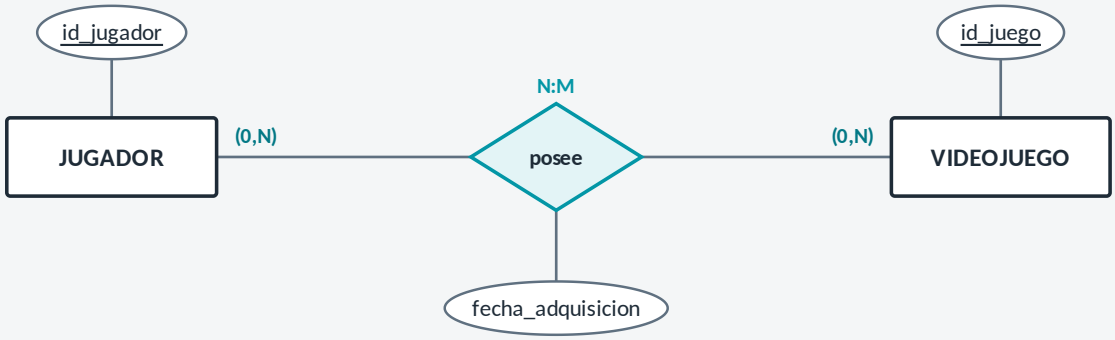
Otro convenio que puedes encontrar

Algunos libros escriben el par junto a la entidad «que participa» (cuántas veces participa cada ocurrencia de esa entidad), con lo que los pares quedan **cruzados** respecto al nuestro. Los dos describen lo mismo; en este módulo usamos siempre el del **extremo opuesto**. Si consultas otro material, fíjate primero en qué convenio sigue.

2.6 Atributos de una relación

A veces un dato no pertenece a ninguna de las entidades, sino a la **combinación** de ambas. En el requisito 5, la fecha en la que un jugador adquirió un juego:

- No es un atributo de JUGADOR: un jugador tiene muchos juegos, cada uno adquirido en una fecha.
- No es un atributo de VIDEOJUEGO: un juego lo tienen muchos jugadores, cada uno desde una fecha.
- Es un atributo de la relación **posee**: depende de la pareja (jugador, juego).



Relación N:M con un atributo propio.

Error frecuente

Colocar `fecha_adquisicion` dentro de VIDEOJUEGO. Solo podrías guardar una fecha por juego, ¡para todos los jugadores a la vez!

2.7 Relaciones reflexivas

Una relación **reflexiva** asocia una entidad consigo misma. Como las dos «patas» salen de la misma entidad, hay que indicar el **papel** (rol) que desempeña cada una; si no, el diagrama es ambiguo.



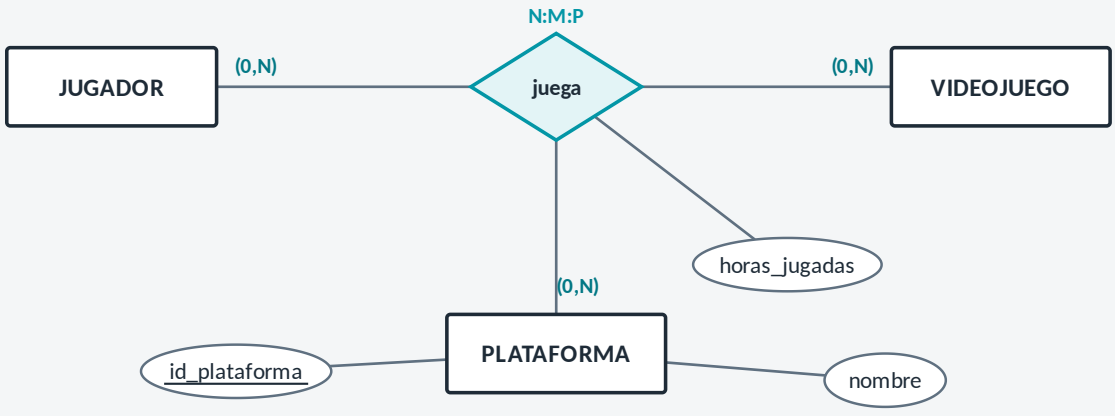
Requisitos 7 y 8: un jugador tiene muchos amigos (N:M) y un juego es secuela, como mucho, de otro (1:1).

En una reflexiva se aplica el mismo convenio, pero pensando en **papeles**: el par que está junto a una pata cuenta las ocurrencias que desempeñan **ese papel**.

- **es_amigo_de** es N:M: un jugador puede tener muchos amigos y ser amigo de muchos.
- **es_secuela_de** es 1:1 con papeles *original* y *secuela*: Hades II es la secuela de Hades; Hades es el original. Los dos pares son `(0,1)`: un juego tiene como mucho una secuela y es secuela, como mucho, de un original; y el mínimo es 0 porque la mayoría de los juegos no tienen secuela ni son secuela de nada.

2.8 Relaciones ternarias

Una relación **ternaria** asocia tres entidades a la vez. En el requisito 6 interesa saber cuántas horas ha jugado **cada jugador a cada juego en cada plataforma**: el dato `horas_jugadas` depende de los tres a la vez.



La ternaria juega: NoctisRJ ha jugado 40 h a Hades en PC y 12 h a Hades en Switch 2.

¿Por qué no tres relaciones binarias?

Podríamos intentar sustituirla por tres binarias (jugador–juego, jugador–plataforma, juego–plataforma), pero perderíamos información. Saber que NoctisRJ tiene Hades, que NoctisRJ juega en Switch 2 y que Hades está disponible en Switch 2 **no** nos dice que NoctisRJ haya jugado a Hades en Switch 2, ni cuántas horas. Por eso la ternaria es necesaria.

Cardinalidad en una ternaria

En una ternaria se sigue la misma idea del extremo opuesto: **se fijan las otras dos** entidades, se pregunta cuántas ocurrencias de la tercera puede haber, y el par se escribe **junto a esa tercera**:

- Fijado un jugador y un juego, ¿en cuántas plataformas puede haber jugado? En varias → N.
- Fijado un jugador y una plataforma, ¿a cuántos juegos? A varios → N.
- Fijado un juego y una plataforma, ¿cuántos jugadores? Muchos → N.

La relación es **N:M:P**. Si alguna de las respuestas fuera «uno como máximo», ese lado tendría un 1 y la clave de la tabla resultante cambiaría (lo verás en el ejercicio 6).

2.9 El diagrama de PixelVault al final de la reunión 1

Juntando todas las piezas, este es el diagrama E/R que resulta de los requisitos 1 a 9. Para que se lea mejor, los atributos de las entidades no se dibujan (están en las figuras anteriores); sí se dibujan los atributos de las relaciones.

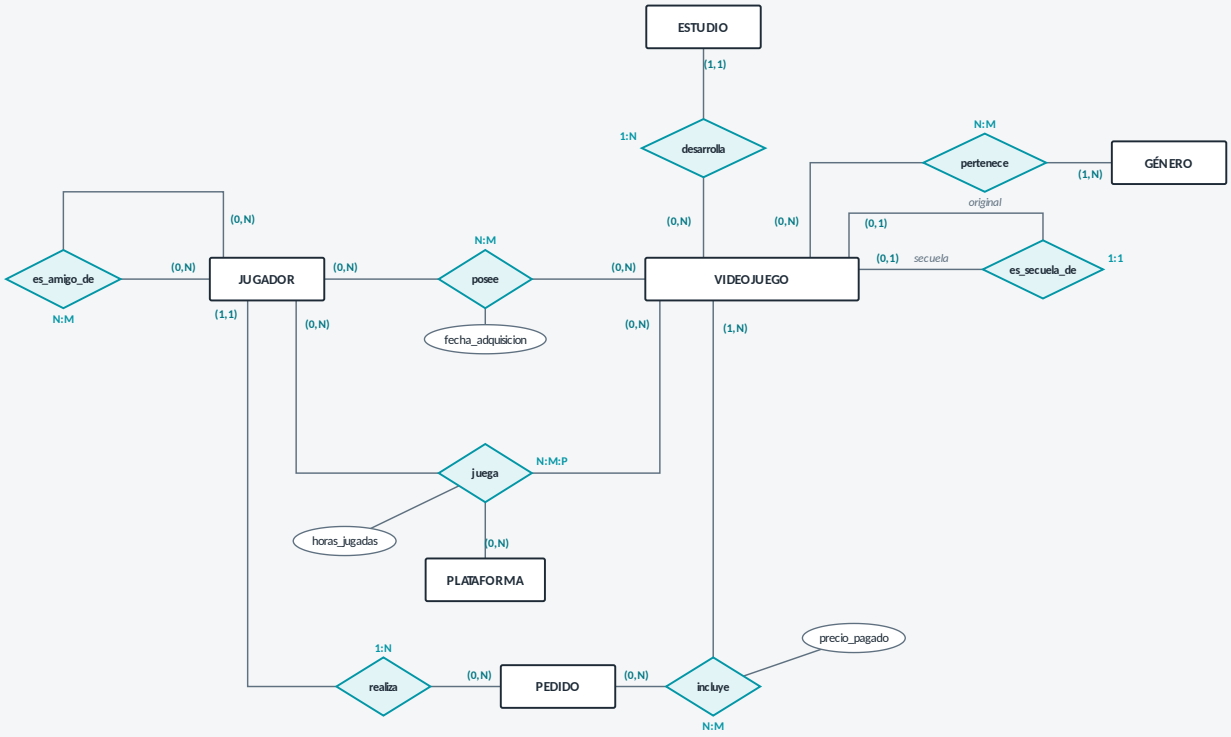


Diagrama E/R de PixelVault tras la reunión 1. En el apartado 3 lo ampliaremos. Abrir a tamaño completo.

La relación **incluye** es N:M con el atributo `precio_pagado`: un pedido incluye uno o varios juegos, por eso `(1,N)` junto a VIDEOJUEGO, y un juego puede estar en muchos pedidos o en ninguno, por eso `(0,N)` junto a PEDIDO. El precio pagado es de la relación, no del juego, porque el mismo juego se vende a precios distintos según el día.

Comprueba tu diagrama

Si en el apartado anterior subrayaste las entidades del caso, compáralas con este diagrama. ¿Habías puesto GÉNERO como atributo? ¿Y `horas_jugadas` dentro de VIDEOJUEGO? Son los dos fallos más habituales.

Resumen

- **Entidad**: algo de lo que interesa guardar datos (rectángulo). **Atributo**: una propiedad (elipse). **Relación**: una asociación entre entidades (rombo).
- Los atributos pueden ser simples, compuestos, multivaluados, derivados o identificadores. La **clave primaria** se subraya.
- La **cardinalidad (mín, máx)** se escribe en el **extremo opuesto**: junto a B va cuántas B corresponden a una A. Los máximos dan el tipo **1:1, 1:N o N:M**.
- Un dato que depende de la combinación de entidades es un **atributo de la relación**.
- Las **reflexivas** necesitan roles; las **ternarias** no se pueden sustituir por tres binarias.

UD 2 · APARTADO 3

Entidades débiles y modelo E/R ampliado

El modelo E/R original de Chen se queda corto para algunas situaciones muy frecuentes: entidades que no pueden existir sin otra, entidades que son «tipos de» otra, o relaciones que se apoyan en otra relación. El **modelo E/R ampliado** (o extendido) añade construcciones para representarlas. En este apartado las aplicamos a la reunión 2 de PixelVault.

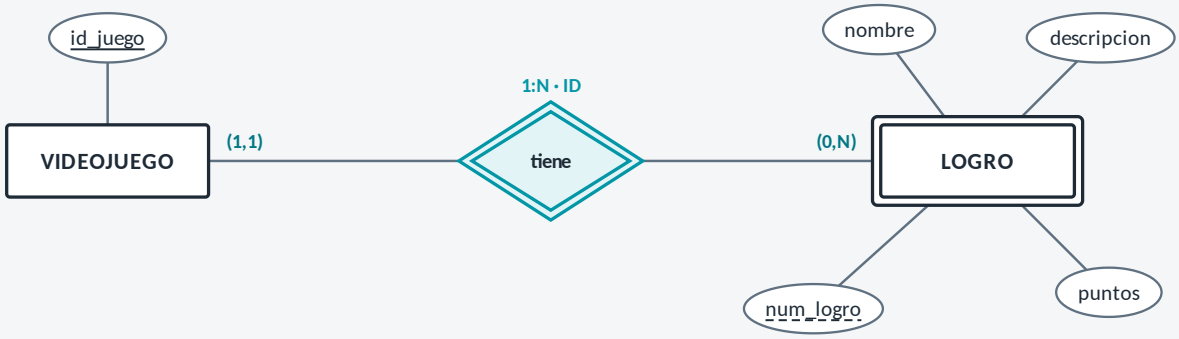
3.1 Entidades débiles CE 6a

Una **entidad débil** es la que depende de otra entidad, llamada **fuerte** o propietaria. Se dibuja con **doble rectángulo**, y la relación que la une a su entidad fuerte, con **doble rombo**. Hay dos tipos de dependencia.

Dependencia de identificación: LOGRO

El requisito 12 dice que los logros se numeran **dentro de cada juego**: Elden Ring tiene su logro 1, su logro 2…; y Hades también tiene su logro 1, su logro 2… El número de logro por sí solo no identifica nada: el logro 1 de Elden Ring y el logro 1 de Hades son distintos.

Para identificar un logro necesitamos **el juego y el número**. LOGRO es una entidad débil **por identificación** (se marca `ID` junto a la relación). Su atributo `num_logro` es un **discriminador** o clave parcial y se subraya con línea discontinua.



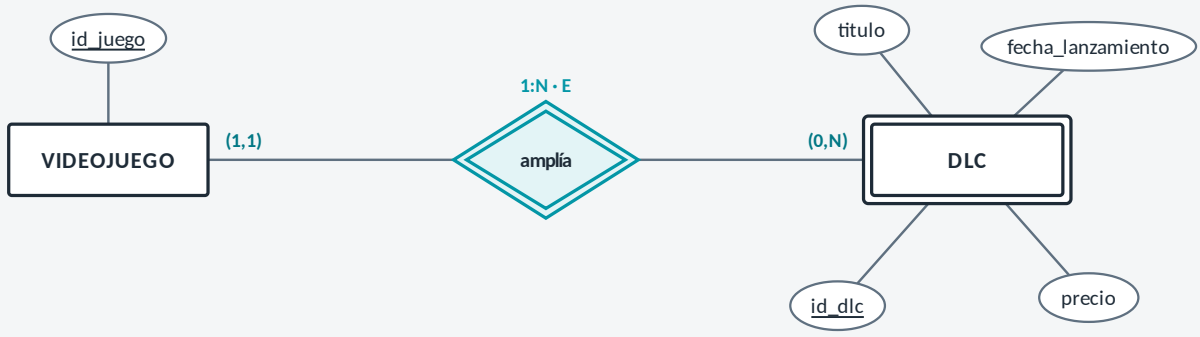
LOGRO es débil por identificación: su clave completa será (id_juego, num_logro).

id_juego	num_logro	nombre	puntos
4 (Elden Ring)	1	Primera victoria	10
4 (Elden Ring)	2	Explorador de las Tierras Intermedias	30
6 (Hades)	1	Primera victoria	10

Fíjate en que «1 · Primera victoria» aparece dos veces sin ningún problema: son logros de juegos distintos.

Dependencia de existencia: DLC

El requisito 13 dice que cada DLC tiene **un código propio** (se identifica solo) pero **no puede existir sin su juego**: si el juego se retira, se retiran sus DLC. DLC es una entidad débil **por existencia** (`E`). Su clave (`id_dlc`) se subraya con línea continua porque es completa.



DLC es débil por existencia: tiene clave propia, pero no existe sin su juego. Por ejemplo, Shadow of the Erdtree (2024) amplía Elden Ring.

	Dependencia de identificación	Dependencia de existencia
¿Existe sin la fuerte?	No	No
¿Se identifica sola?	**No**: necesita la clave de la fuerte	**Sí**: tiene clave propia
Clave en el diagrama	Discriminador (subrayado discontinuo)	Clave normal (subrayado continuo)
Ejemplo	LOGRO (logro 1 *de* Elden Ring)	DLC (Phantom Liberty, que amplía Cyberpunk 2077)

Cardinalidad de la débil

En la relación de dependencia, junto a la **entidad fuerte** aparece siempre `(1,1)` : todo logro pertenece a exactamente un juego, y todo DLC, a exactamente uno. Junto a la débil va cuántas tiene cada fuerte, normalmente `(0,N)` o `(1,N)` . Toda dependencia de identificación implica también dependencia de existencia (si no sabes de qué juego es un logro, no sabes qué logro es).

Error frecuente

Considerar débil a PEDIDO porque «un pedido no tiene sentido sin su jugador». PEDIDO tiene clave propia (`num_pedido`) y, por motivos contables, un pedido debe conservarse aunque el jugador se dé de baja. Que cada pedido esté ligado obligatoriamente a un jugador (el `(1,1)` junto a JUGADOR en *realiza*) **no** convierte a PEDIDO en débil.

3.2 Generalización y especialización CE 6a

Una **jerarquía** (relación *ES UN*) representa que varias entidades son tipos de otra más general. La entidad general se llama **supertipo** (superclase) y las particulares, **subtipos** (subclases). Se dibuja con un **triángulo** con el texto *ES UN*.

- **Generalización** (de abajo arriba): partimos de JUGADOR y DESARROLLADOR, vemos que comparten atributos y los reunimos en USUARIO.
- **Especialización** (de arriba abajo): partimos de VIDEOJUEGO y lo dividimos en tipos porque cada uno tiene atributos o relaciones propias.

El resultado se dibuja igual; lo que cambia es cómo se llega a él.

Herencia

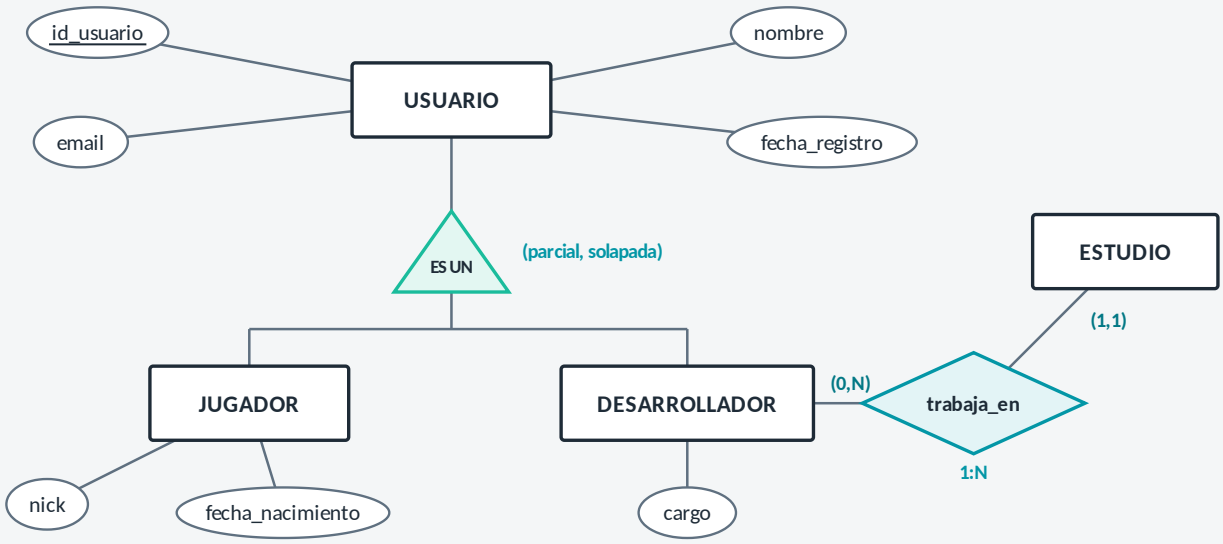
Los subtipos **heredan** todos los atributos y relaciones del supertipo. En el diagrama, cada atributo se pone **una sola vez**, en el nivel más alto donde tenga sentido: `email` va en USUARIO, no se repite en JUGADOR ni en DESARROLLADOR. Los subtipos solo llevan lo que les es **propio**.

Tipos de jerarquía

Se clasifican según dos criterios independientes, que se anotan junto al triángulo:

Criterio	Opción	Significado
Cobertura	Total	Toda ocurrencia del supertipo pertenece a algún subtipo.
	Parcial	Puede haber ocurrencias del supertipo que no pertenecen a ningún subtipo.
Solapamiento	Exclusiva (disjunta)	Una ocurrencia pertenece como máximo a un subtipo.
	Solapada (inclusiva)	Una ocurrencia puede pertenecer a varios subtipos a la vez.

USUARIO: parcial y solapada

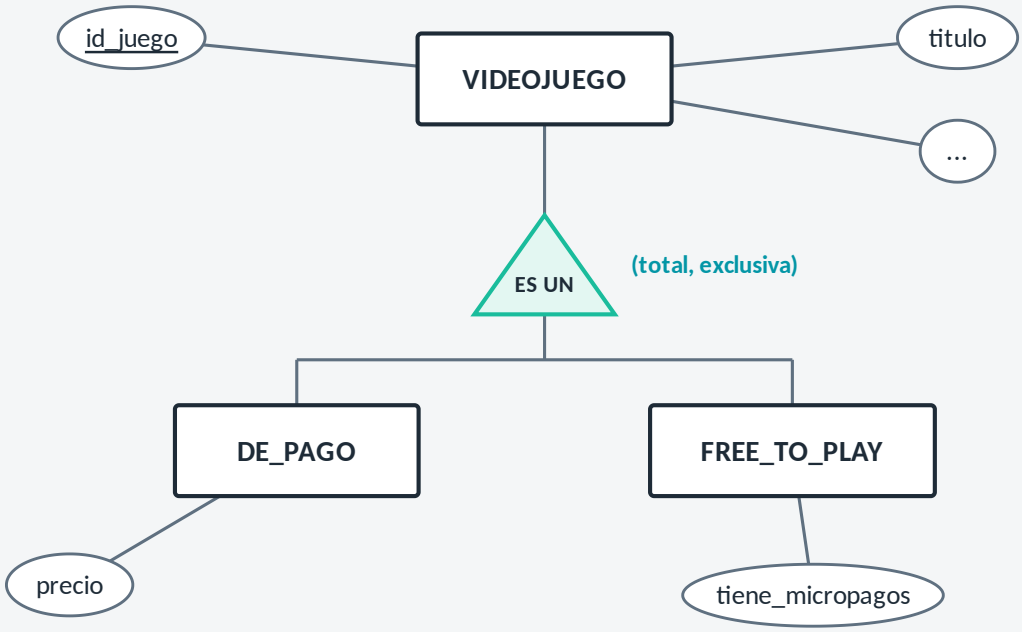


Requisito 10. Solo DESARROLLADOR participa en trabaja_en: esa relación es propia del subtipo.

- **Parcial**: hay usuarios (el personal de soporte) que no son ni jugadores ni desarrolladores.
- **Solapada**: una misma persona puede ser jugador y desarrollador a la vez.

Esta jerarquía cambia algo del apartado 2: JUGADOR ya no necesita su propio `id_jugador` ni su `email`, porque los hereda de USUARIO. Su identificador es ahora `id_usuario`.

VIDEOJUEGO: total y exclusiva



Requisito 11. Todo juego es de pago o free-to-play, y nunca las dos cosas.

- **Total**: todo juego es de uno de los dos tipos.
- **Exclusiva**: ningún juego es de los dos tipos a la vez.

La ventaja de especializar no son solo los atributos (`precio` , `tiene_micropagos`), sino **las relaciones**. El requisito 11 dice que solo los juegos de pago se venden en pedidos. Con la jerarquía, basta con conectar la relación *incluye* a **DE_PAGO** en lugar de a VIDEOJUEGO: el diagrama impide por sí mismo meter Fortnite en un pedido.

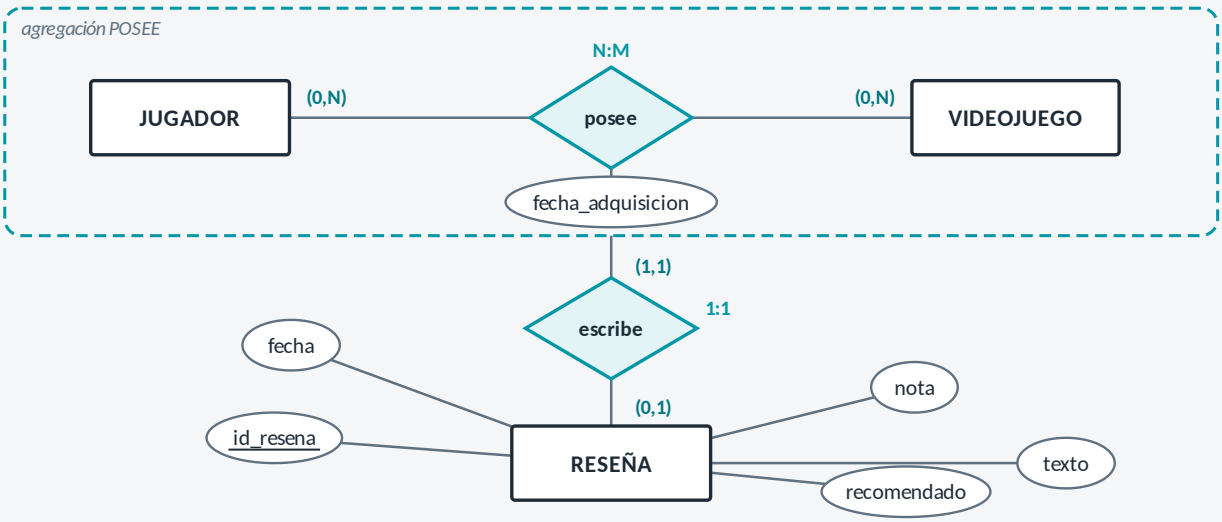
¿Cuándo merece la pena una jerarquía?

Solo si los subtipos tienen **atributos propios** o **relaciones propias**. Si lo único que distingue a los tipos es el nombre, basta con un atributo `tipo` en la entidad.

3.3 Agregación CE 6a

En el modelo E/R básico, una relación solo puede unir entidades. Pero el requisito 14 dice que un jugador solo puede reseñar **los juegos que tiene en su biblioteca**: la reseña no se asocia a un jugador cualquiera ni a un juego cualquiera, sino a una **posesión** concreta (una pareja jugador–juego que ya existe en *posee*).

La **agregación** permite tratar una relación, junto con sus entidades, como si fuera una entidad de nivel superior, para relacionarla con otras. Se dibuja con un **recuadro** alrededor de la relación y sus entidades.



Cada posesión tiene como mucho una reseña: (0,1) junto a RESEÑA. Cada reseña corresponde exactamente a una posesión: (1,1) junto a la agregación.

Con esta construcción el diagrama refleja dos reglas del requisito 14:

- **Solo se reseña lo que se posee**: la reseña cuelga de la agregación, no de JUGADOR ni de VIDEOJUEGO por separado.
- **Una reseña por juego y jugador**: junto a RESEÑA aparece `(0,1)`, es decir, cada posesión tiene como mucho una reseña.

¿Y si no usáramos agregación?

Una alternativa habitual sería relacionar RESEÑA con JUGADOR y con VIDEOJUEGO por separado. Funcionaría, pero el diagrama ya no diría que el jugador tiene que poseer el juego; habría que documentarlo como restricción aparte. La agregación expresa más con el propio dibujo.

3.4 El diagrama completo de PixelVault

Este es el diagrama E/R definitivo. **En verde**, lo que ha añadido la reunión 2. Como antes, solo se dibujan los atributos de las relaciones; los de las entidades están en el diccionario de datos de la sección siguiente.

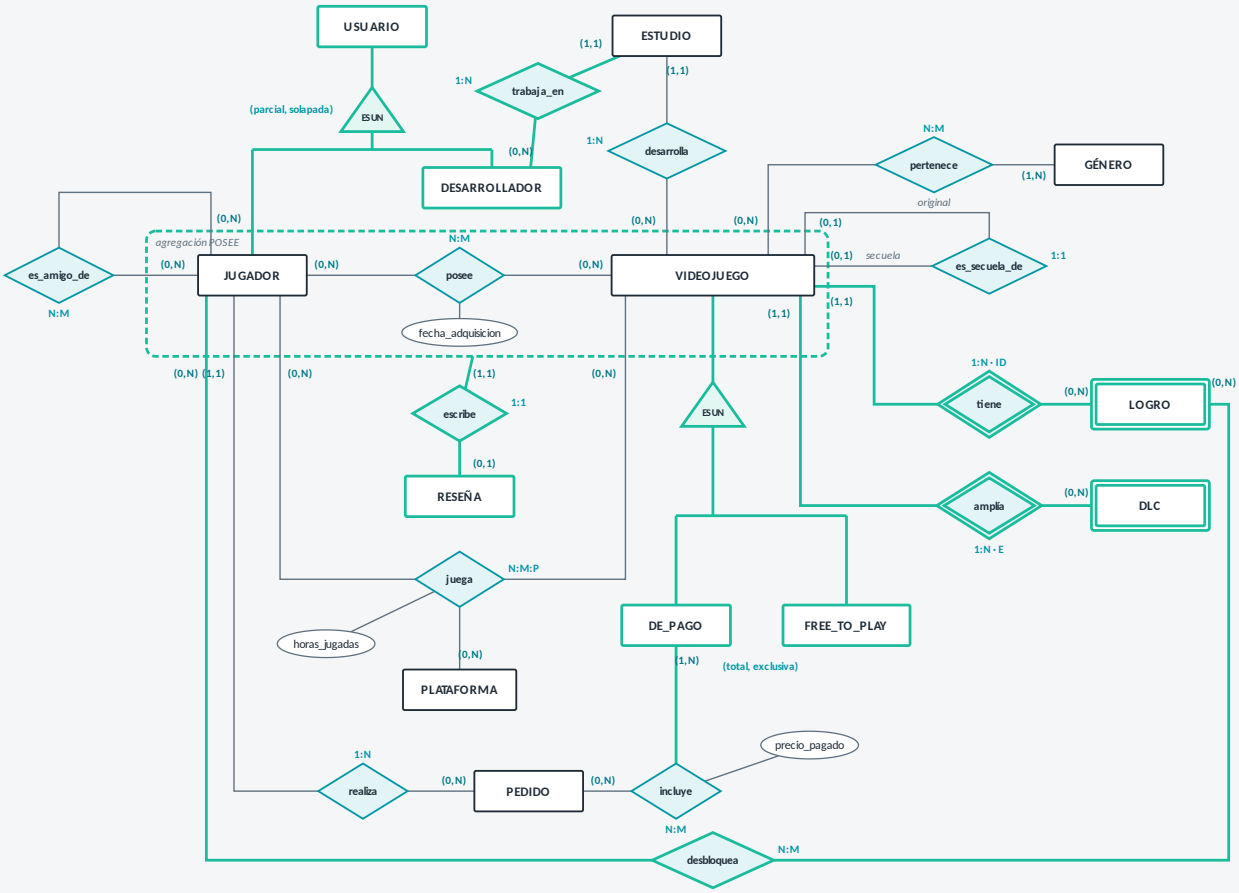


Diagrama E/R completo de PixelVault. Si la pantalla es estrecha, desplázalo en horizontal. Abrir a tamaño completo.

Fíjate en los cambios respecto al diagrama del apartado 2:

- JUGADOR ahora es un subtipo de USUARIO.
- *incluye* ya no conecta con VIDEOJUEGO, sino con su subtipo DE_PAGO.
- *posee* está dentro de una agregación, de la que cuelga *escribe*.
- Han aparecido dos entidades débiles (LOGRO y DLC) y la relación *desbloquea* entre JUGADOR y la débil LOGRO.

3.5 Diccionario de datos

Un diagrama E/R se acompaña siempre de un **diccionario de datos**: la lista de entidades y relaciones con todos sus atributos. Es lo que usaremos en el apartado 5 para pasar a tablas.

Entidades

Convenio: **clave** en negrita, `compuesto(c1, c2...)`, `{multivaluado}`, `/derivado`.

Entidad	Atributos
USUARIO	**id_usuario**, email, nombre, apellidos, fecha_registro
JUGADOR *(subtipo de USUARIO)*	nick, fecha_nacimiento, direccion(calle, numero, cp, ciudad, pais), /edad
DESARROLLADOR *(subtipo de USUARIO)*	cargo
ESTUDIO	**id_estudio**, nombre, pais, web
VIDEOJUEGO	**id_juego**, titulo, fecha_lanzamiento, pegi, {idioma}, /nota_media
DE_PAGO *(subtipo de VIDEOJUEGO)*	precio
FREE_TO_PLAY *(subtipo de VIDEOJUEGO)*	tiene_micropagos
GÉNERO	**id_genero**, nombre
PLATAFORMA	**id_plataforma**, nombre, fabricante
PEDIDO	**num_pedido**, fecha
LOGRO *(débil por identificación)*	**num_logro** *(discriminador)*, nombre, descripcion, puntos
DLC *(débil por existencia)*	**id_dlc**, titulo, precio, fecha_lanzamiento
RESEÑA	**id_resena**, nota, texto, fecha, recomendado

Relaciones

Relación	Participantes y cardinalidad	Tipo	Atributos
desarrolla	ESTUDIO (1,1) — VIDEOJUEGO (0,N)	1:N	—
trabaja_en	ESTUDIO (1,1) — DESARROLLADOR (0,N)	1:N	—
pertenece	VIDEOJUEGO (0,N) — GÉNERO (1,N)	N:M	—
es_secuela_de	VIDEOJUEGO *original* (0,1) — VIDEOJUEGO *secuela* (0,1)	1:1	—
posee	JUGADOR (0,N) — VIDEOJUEGO (0,N)	N:M	fecha_adquisicion
juega	JUGADOR (0,N) — VIDEOJUEGO (0,N) — PLATAFORMA (0,N)	N:M:P	horas_jugadas
es_amigo_de	JUGADOR (0,N) — JUGADOR (0,N)	N:M	—

Relación	Participantes y cardinalidad	Tipo	Atributos
realiza	JUGADOR (1,1) — PEDIDO (0,N)	1:N	—
incluye	PEDIDO (0,N) — DE_PAGO (1,N)	N:M	precio_pagado
tiene *(ID)*	VIDEOJUEGO (1,1) — LOGRO (0,N)	1:N	—
amplía *(E)*	VIDEOJUEGO (1,1) — DLC (0,N)	1:N	—
desbloquea	JUGADOR (0,N) — LOGRO (0,N)	N:M	—
escribe	agregación POSEE (1,1) — RESEÑA (0,1)	1:1	—

Sobre la lectura de las tablas

En la tabla, cada par va junto a la entidad a la que acompaña en el diagrama, es decir, en el **extremo opuesto**: en *tiene*, LOGRO (0,N) significa que un juego tiene de 0 a N logros, y VIDEOJUEGO (1,1), que cada logro es de exactamente un juego. En la ternaria *juega*, cada par se obtiene fijando las otras dos entidades.

Resumen

- Una **entidad débil** depende de una fuerte: **por existencia** (tiene clave propia, como DLC) o **por identificación** (necesita la clave de la fuerte, como LOGRO).
- Una **jerarquía** ES UN reúne atributos y relaciones comunes en un supertipo; puede ser **total o parcial** y **exclusiva o solapada**.
- Los subtipos heredan todo del supertipo; en ellos solo se dibuja lo **propio**.
- La **agregación** permite relacionar una relación con otra entidad (la reseña cuelga de la posesión).
- El diagrama se acompaña de un **diccionario de datos** con todos los atributos.

UD 2 · APARTADO 4

El modelo relacional

El diagrama E/R describe la realidad; ahora hay que llevarlo a un modelo que un gestor como Oracle pueda implementar. Ese es el **modelo relacional**, propuesto por Edgar F. Codd (IBM) en 1970 y base de casi todos los SGBD empresariales actuales. En este apartado aprendemos su vocabulario y sus reglas antes de hacer la transformación en el apartado 5.

4.1 La idea: todo son tablas CE 6b CE 6c

En el modelo relacional la información se organiza en **relaciones**, que se representan como **tablas** de dos dimensiones. Cada fila guarda una ocurrencia y cada columna, un dato.

id_juego	titulo	fecha_lanzamiento	pegi	id_estudio
1	Grand Theft Auto V	2013-09-17	18	10
2	Baldur's Gate 3	2023-08-03	18	11
6	Hades	2020-09-17	12	15
11	Hades II	2025-09-25	12	15

⚠ **Relación ≠ relación**

Cuidado con la palabra: en el modelo **E/R**, una *relación* es un rombo que asocia entidades. En el modelo **relacional**, una *relación* es una **tabla**. El nombre del modelo viene de este segundo significado (del concepto matemático de relación), no de que las tablas estén relacionadas.

4.2 Terminología

Cada concepto tiene un nombre formal, uno habitual y uno «de fichero». Conviene conocer los tres, porque los verás mezclados en la documentación.

Término formal	Término habitual	En ficheros	En la tabla de ejemplo
Relación	Tabla	Fichero	VIDEOJUEGO
Tupla	Fila, registro	Registro	La fila de Hades
Atributo	Columna, campo	Campo	`titulo`
Dominio	Tipo de dato + valores válidos	—	`pegi` ∈

Término formal	Término habitual	En ficheros	En la tabla de ejemplo
Grado	Número de columnas	—	5
Cardinalidad	Número de filas	—	4 (cambia con el tiempo)

Esquema de una relación

> Su «molde»: el nombre de la tabla y la lista de sus atributos. Cambia poco. Por ejemplo, `VIDEOJUEGO(id_juego, titulo, fecha_lanzamiento, pegi, id_estudio)`.

Instancia (o extensión) de una relación

> Las filas que contiene en un momento dado. Cambia cada vez que se inserta, modifica o borra.

> **Cardinalidad, otra vez**
>
> La *cardinalidad de una tabla* (número de filas) no tiene nada que ver con la *cardinalidad de una relación E/R* (1:N, N:M...). Es otro término que se repite con distinto significado.

4.3 Propiedades de una tabla relacional

Una tabla del modelo relacional no es una hoja de cálculo cualquiera. Debe cumplir estas propiedades:

1. **No hay dos filas iguales.** Siempre hay algo (la clave primaria) que las distingue.
2. **El orden de las filas no importa.** Las filas no tienen «posición»; si se necesita un orden, se pide en la consulta (`ORDER BY`).
3. **El orden de las columnas no importa.** Se accede a ellas por nombre, no por posición.
4. **Cada celda contiene un único valor atómico.** No se permiten listas ni grupos repetidos dentro de una celda. Por ejemplo, no se puede guardar `'español, inglés, francés'` en una columna `idiomas`. Esta propiedad es la **primera forma normal**, que retomaremos en el apartado 7.
5. **Todos los valores de una columna pertenecen al mismo dominio.**

4.4 Claves CE 6e

Las claves son la pieza que hace funcionar el modelo: identifican las filas y conectan unas tablas con otras.

Superclave

> Cualquier conjunto de columnas cuyos valores no se repiten en dos filas. Por ejemplo, `(id_juego, titulo)` es superclave de VIDEOJUEGO, pero le sobra `titulo`.

Clave candidata

> Una superclave **mínima**: si le quitas cualquier columna, deja de identificar. En JUGADOR son claves candidatas `id_usuario` y `nick`.

Clave primaria (PK, *primary key*)

> La clave candidata elegida para identificar las filas. **No puede repetirse ni ser nula**. Se subraya.

Clave alternativa (UK, *unique key*)

> Cada clave candidata no elegida como primaria. No puede repetirse. En Oracle se declara con `UNIQUE`.

Clave ajena (FK, *foreign key*)

> Una columna (o conjunto de columnas) cuyos valores deben coincidir con los de la clave primaria (o alternativa) de **otra tabla**, o de la misma. Es el mecanismo que implementa las relaciones. En VIDEOJUEGO, `id_estudio` es clave ajena hacia ESTUDIO: solo admite identificadores de estudios que existan.

ESTUDIO

id_estudio	nombre	pais
10	Rockstar North	Reino Unido
11	Larian Studios	Bélgica
15	Supergiant Games	EE. UU.

Hades tiene `id_estudio = 15`, y 15 es la clave primaria de Supergiant Games en ESTUDIO. Así, sin repetir el nombre ni el país del estudio en cada juego, sabemos quién lo desarrolló.

Clave simple y clave compuesta

Una clave es **simple** si tiene una sola columna (`id_juego`) y **compuesta** si tiene varias. La clave primaria de LOGRO, por ejemplo, es compuesta: `(id_juego, num_logro)`. Una clave ajena hacia una tabla con clave compuesta también tiene que ser compuesta: tiene que llevar **todas** las columnas.

4.5 El valor NULL

NULL es un marcador especial que indica **ausencia de valor**. Puede significar dos cosas distintas:

- El valor **no se conoce** (no sabemos la web de un estudio).
- El valor **no es aplicable** (un juego que no es secuela de nada no tiene juego original).

> ⚠ **NULL no es cero ni cadena vacía**
>
> `NULL` no es `0`, ni `''`, ni un espacio. Tampoco es igual a otro `NULL`: en SQL, `NULL = NULL` no es verdadero, sino **desconocido**. Por eso en Oracle se pregunta con `IS NULL` y no con `= NULL`. Lo verás en la UD 3.

Las reglas del diseño lógico sobre los nulos son:

Columna	¿Admite NULL?
Clave primaria (o cualquier parte de ella)	**Nunca** (regla de integridad de entidad)
Clave ajena de una 1:N cuyo par en el lado 1 tiene mínimo 1, `(1,1)`	No: `NOT NULL`
Clave ajena de una 1:N cuyo par en el lado 1 tiene mínimo 0, `(0,1)`	Sí
Resto de columnas	Según los requisitos (el correo, obligatorio; la web, opcional)

4.6 Cómo escribimos los esquemas en estos apuntes

Para escribir un esquema relacional en texto usaremos este convenio, el mismo de los supuestos de clase:

```
VIDEOJUEGO(id_juego, titulo, fecha_lanzamiento, pegi, id_estudio → ESTUDIO NOT NULL)
LOGRO(id_juego → VIDEOJUEGO, num_logro, nombre, descripcion, puntos)
```

- La **clave primaria** va subrayada.
- Las **claves ajenas** van en cursiva con una flecha hacia la tabla referenciada.
- Una columna puede ser las dos cosas a la vez (en LOGRO, `id_juego` forma parte de la PK y además es FK).
- Se añade `NOT NULL`, `UNIQUE` o `NULL` cuando sea importante para el diseño.

Resumen

- En el modelo relacional todo se guarda en **tablas** (relaciones) formadas por **filas** (tuplas) y **columnas** (atributos).
- Cada celda tiene un **valor atómico**, no hay filas repetidas y el orden no importa.
- La **clave primaria** identifica cada fila y no admite nulos; las **claves ajenas** enlazan tablas.
- **NULL** significa ausencia de valor: desconocido o no aplicable. No es cero ni cadena vacía.

UD 2 · APARTADO 5

Del diagrama E/R al esquema relacional

Este es el paso central del diseño lógico: aplicar unas **reglas de transformación** al diagrama E/R para obtener las tablas, sus columnas y sus claves. Las reglas son mecánicas, así que si el diagrama está bien hecho, el esquema sale casi solo. Las veremos una a una, aplicadas al diagrama completo de PixelVault.

Antes de empezar: nombres válidos en Oracle

Los nombres de tablas y columnas en Oracle no deberían llevar tildes, eñes ni espacios. Por eso, al pasar a tablas, GÉNERO se convierte en `GENERO` y RESEÑA en `RESENA`. Usaremos mayúsculas para las tablas y minúsculas con guion bajo para las columnas.

5.1 Entidades fuertes CE 6b CE 6c CE 6e

Regla 1 · Entidad

Cada entidad fuerte se convierte en una **tabla** con el mismo nombre. Sus atributos simples son las **columnas**, su identificador es la **clave primaria** y sus claves alternativas se marcan como **UNIQUE**.

```
ESTUDIO(id_estudio, nombre, pais, web)
GENERO(id_genero, nombre UNIQUE)
PLATAFORMA(id_plataforma, nombre, fabricante)
```

5.2 Atributos especiales

Regla 2 · Atributo compuesto

Se sustituye por sus **componentes**, cada uno como una columna. El atributo compuesto como tal desaparece.

La dirección de JUGADOR se convierte en `calle, numero, cp, ciudad, pais`.

Regla 3 · Atributo multivaluado

Se crea una **tabla nueva** con la clave primaria de la entidad (que además es clave ajena) y una columna para el valor. La clave primaria de la tabla nueva es **la combinación de las dos**.

```
IDIOMA_JUEGO(id_juego → VIDEOJUEGO, idioma)
```

id_juego	idioma
2 (Baldur's Gate 3)	español
2 (Baldur's Gate 3)	inglés
2 (Baldur's Gate 3)	francés
6 (Hades)	español

Error frecuente

Guardar los idiomas en una sola columna de VIDEOJUEGO: `idiomas = 'español, inglés, francés'`. Rompe la atomicidad (primera forma normal) y hace imposible consultar con sencillez qué juegos están en francés.

Regla 4 · Atributo derivado

No se guarda. Se calcula cuando se necesita, con una consulta o una vista. `edad` y `nota_media` no aparecen en las tablas.

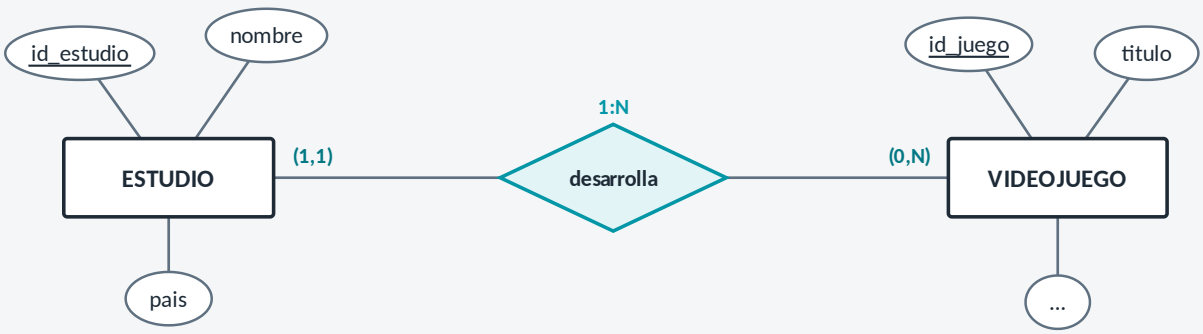
5.3 Relaciones 1:N: propagación de clave CE 6d

Regla 5 · Relación 1:N → propagación de clave

No se crea tabla. La clave primaria de la entidad del lado **1** se **propaga** a la tabla de la entidad del lado **N**, donde se convierte en **clave ajena**. Si el par que aparece en el lado 1 tiene mínimo 1, `(1,1)`, la clave ajena es **NOT NULL**; si tiene mínimo 0, `(0,1)`, admite nulos. Los atributos de la relación, si los hay, se propagan con ella.

A este mecanismo se le llama **propagación de clave**: la clave «viaja» del lado 1 al lado N. Con el convenio del extremo opuesto se lee directamente en el diagrama:

- El lado **1** es el que tiene el máximo 1 **escrito junto a él**; el lado **N**, el que tiene la N.
- La clave viaja **del lado 1 al lado N**.
- El **mínimo del par del lado 1** decide si la clave ajena admite nulos: en *desarrolla*, junto a ESTUDIO hay `(1,1)`, así que todo videojuego debe tener estudio y `id_estudio` es `NOT NULL`.



La clave de ESTUDIO (lado 1) se propaga a VIDEOJUEGO (lado N).

ESTUDIO(**id_estudio**, nombre, pais)
VIDEOJUEGO(**id_juego**, titulo, ..., *id_estudio → ESTUDIO* NOT NULL) — *clave propagada desde ESTUDIO*

Las tres relaciones 1:N «normales» de PixelVault quedan así:

Relación (en el orden 1 → N)	Se propaga la clave de...	...a la tabla	Clave ajena	¿Nula?
ESTUDIO (1,1) *desarrolla* VIDEOJUEGO (0,N)	ESTUDIO	VIDEOJUEGO	`id_estudio → ESTUDIO`	NOT NULL
ESTUDIO (1,1) *trabaja_en* DESARROLLADOR (0,N)	ESTUDIO	DESARROLLADOR	`id_estudio → ESTUDIO`	NOT NULL
JUGADOR (1,1) *realiza* PEDIDO (0,N)	JUGADOR	PEDIDO	`id_jugador → JUGADOR`	NOT NULL

¿Por qué hacia el lado N y no al revés?

Porque cada fila del lado N se relaciona con **una sola** fila del lado 1, y una columna solo puede guardar un valor. Un juego apunta a *un* estudio; un estudio no podría guardar *sus* juegos en una sola columna (rompería la primera forma normal).

Error frecuente

Propagar la clave al revés (poner una columna `id_juego` en ESTUDIO). Fíjate siempre en dónde está la N: **la clave viaja hacia la N**.

La propagación de clave no es exclusiva de las 1:N: también se usa en las **1:1** (apartado 5.5), en las **reflexivas 1:N o 1:1** (5.6) y en las **entidades débiles**, cuya clave se forma propagando la de la fuerte (5.8). En cambio, las **N:M** no se pueden resolver propagando claves y necesitan una tabla nueva.

5.4 Relaciones N:M

Regla 6 · Relación N:M

Se crea una **tabla nueva** con las claves primarias de las dos entidades como claves ajenas. La **clave primaria** de la nueva tabla es la **combinación de ambas**. Los atributos de la relación son columnas de la nueva tabla.

```
JUEGO_GENERO(id_juego → VIDEOJUEGO, id_genero → GENERO)
BIBLIOTECA(id_jugador → JUGADOR, id_juego → VIDEOJUEGO, fecha_adquisicion)
LINEA_PEDIDO(num_pedido → PEDIDO, id_juego → JUEGO_PAGO, precio_pagado)
```

La relación *posee* se ha convertido en la tabla `BIBLIOTECA`: es habitual dar a la tabla un nombre más natural que el verbo del rombo.

¿Y si la combinación puede repetirse?

La PK (jugador, juego) impide que un jugador tenga dos veces el mismo juego, que es justo lo que queremos. Pero si una combinación pudiera repetirse (por ejemplo, un jugador que alquila el mismo juego varias veces en fechas distintas), la **fecha** tendría que formar parte de la clave primaria. Piensa siempre si el par puede repetirse.

5.5 Relaciones 1:1

Regla 7 · Relación 1:1

No se crea tabla. Se hace una **propagación de clave** en un solo sentido: la clave primaria de una entidad pasa como **clave ajena con UNIQUE** a la tabla de la otra. El sentido se elige mirando los mínimos de los dos pares:

- Si un par es `(1,1)` y el otro `(0,1)`, se propaga la clave de la entidad que tiene **junto a ella el** `(1,1)` hacia la otra tabla, donde la clave ajena es `NOT NULL`. (Ese `(1,1)` dice que toda ocurrencia de la otra entidad tiene obligatoriamente una de esta.)
- Si los dos pares son `(0,1)`, la clave se propaga hacia la tabla donde vaya a haber **menos nulos**, y admite `NULL`.
- Si los dos pares son `(1,1)`, se puede valorar **fusionar** las dos entidades en una sola tabla.

La única 1:1 de PixelVault es la reflexiva *es_secuela_de*, con los dos pares `(0,1)`. Con los dos pares `(0,1)` cualquier sentido es válido, así que elegimos el más natural: propagar la clave del original a cada secuela, porque la pregunta habitual es «¿de qué juego es secuela este?»:

```
VIDEOJUEGO(id_juego, ..., id_juego_original → VIDEOJUEGO UNIQUE NULL)
```

El `UNIQUE` es el que convierte la relación en 1:1: sin él, varios juegos podrían tener el mismo original (sería 1:N).

5.6 Relaciones reflexivas

Regla 8 · Relación reflexiva

Se aplican las mismas reglas que a una binaria, teniendo en cuenta que las dos entidades son la misma tabla. Como aparecen dos columnas que referencian a la misma clave, hay que **darles nombres distintos** según el rol.

- *es_secuela_de* (1:1): la clave se propaga a la propia tabla con otro nombre, `id_juego_original`, como acabamos de ver.
- *es_amigo_de* (N:M): una tabla nueva con dos FK hacia JUGADOR.

```
AMISTAD(id_jugador → JUGADOR, id_amigo → JUGADOR)
```

5.7 Relaciones ternarias

Regla 9 · Relación ternaria

Se crea una **tabla nueva** con las claves primarias de las tres entidades como claves ajenas, más los atributos de la relación. La clave primaria depende de las cardinalidades:

- Si es **N:M:P**, la PK son **las tres** claves ajenas.
- Si una de las entidades tiene cardinalidad **1**, la PK son **las otras dos** (la del lado 1 queda como FK normal).

juega es N:M:P, así que la clave primaria tiene tres columnas:

```
JUEGA(id_jugador → JUGADOR, id_juego → VIDEOJUEGO, id_plataforma → PLATAFORMA,
  horas_jugadas)
```

5.8 Entidades débiles

Regla 10 · Entidad débil

La entidad débil se convierte en una tabla a la que se **propaga** la clave primaria de la fuerte como **clave ajena NOT NULL**, normalmente con **borrado en cascada**.

- **Por identificación**: la PK de la débil es **la PK de la fuerte + el discriminador**.
- **Por existencia**: la PK de la débil es **su propia clave**; la de la fuerte es solo FK.

LOGRO(*id_juego → VIDEOJUEGO*, num_logro, nombre, descripcion, puntos) — *débil por identificación*

DLC(id_dlc, titulo, precio, fecha_lanzamiento, *id_juego → VIDEOJUEGO* NOT NULL) — *débil por existencia*

La relación N:M *desbloquea* se transforma con la regla 6, pero hay que tener en cuenta que la clave de LOGRO tiene **dos** columnas, y las dos viajan juntas:

DESBLOQUEA(*id_jugador → JUGADOR*, id_juego, num_logro)

(id_juego, num_logro) → LOGRO

Error frecuente

Poner en DESBLOQUEA solo `num_logro`. «El logro 1» no identifica ningún logro: hace falta saber de qué juego es. Una clave ajena hacia una PK compuesta tiene que llevar **todas** sus columnas.

5.9 Jerarquías

Regla 11 · Generalización / especialización

Hay tres estrategias. Se elige según el tipo de jerarquía y según cuántos atributos y relaciones propias tengan los subtipos.

Estrategia	Tablas	Cuándo conviene	Inconveniente
A. Una tabla por entidad	Supertipo + una por subtipo; la PK de cada subtipo es a la vez FK al supertipo	Siempre es posible. La mejor si los subtipos tienen relaciones propias.	Para ver todos los datos hay que unir tablas (JOIN).
B. Solo subtipos	Una por subtipo, con los atributos del supertipo repetidos en cada una	Jerarquía **total y exclusiva**, y el supertipo casi no tiene relaciones.	Si es parcial, se pierden ocurrencias; si es solapada, se duplican datos.
C. Una sola tabla	Solo el supertipo, con todos los atributos de los subtipos y una columna `tipo`	Subtipos con **pocos** atributos propios y sin relaciones propias.	Muchos nulos; las restricciones de cada subtipo se controlan con CHECK.

USUARIO → estrategia A

La jerarquía es **parcial** (hay usuarios que no son de ningún subtipo) y **solapada** (se puede ser de los dos). La estrategia B no sirve: perderíamos a los usuarios de soporte y duplicaríamos a quien sea jugador y desarrollador. La C necesitaría dos indicadores y llenaría la tabla de nulos. Elegimos A:

```
USUARIO(id_usuario, email UNIQUE, nombre, apellidos, fecha_registro)
JUGADOR(id_usuario → USUARIO, nick UNIQUE, fecha_nacimiento, calle, numero, cp,
   ciudad, pais)
DESARROLLADOR(id_usuario → USUARIO, cargo, id_estudio → ESTUDIO NOT NULL)
```

Nombres de columna en las tablas que referencian a JUGADOR

La clave de JUGADOR se llama `id_usuario`, pero en las tablas que la referencian (BIBLIOTECA, PEDIDO, AMISTAD...) la llamamos `id_jugador` porque se entiende mejor. Una clave ajena **no tiene que llamarse igual** que la clave a la que apunta; lo que importa es que tenga el mismo dominio.

VIDEOJUEGO → estrategia A

La jerarquía es total y exclusiva, así que las tres estrategias serían posibles. Pero VIDEOJUEGO participa en muchas relaciones (género, estudio, logros, DLC, biblioteca...) y el subtipo DE_PAGO tiene una relación propia (*incluye*). Con la estrategia B habría que duplicar todas esas relaciones; con la C, LINEA_PEDIDO no podría garantizar que el juego es de pago. Elegimos A:

```
VIDEOJUEGO(id_juego, titulo, fecha_lanzamiento, pegi, tipo, id_estudio → ESTUDIO NOT
   NULL, id_juego_original → VIDEOJUEGO UNIQUE NULL)
JUEGO_PAGO(id_juego → VIDEOJUEGO, precio)
JUEGO_F2P(id_juego → VIDEOJUEGO, tiene_micropagos)
```

La columna `tipo` (`'PAGO'` o `'F2P'`) en VIDEOJUEGO ayuda a controlar que la jerarquía es total y exclusiva: cada juego debe tener fila en la tabla de su tipo y solo en ella. Esa comprobación no se puede expresar solo con claves; es una de las restricciones que documentaremos en el apartado 8.

Ampliación: cómo quedaría con la estrategia C

Si PixelVault no vendiera los juegos en pedidos, podríamos usar una sola tabla:

```
VIDEOJUEGO(id_juego, titulo, ..., tipo, precio NULL, tiene_micropagos NULL)
```

Con una restricción `CHECK` que obligue a que los juegos de tipo `'PAGO'` tengan precio y no tengan `tiene_micropagos`, y al revés. Es más simple de consultar, pero los nulos y las comprobaciones crecen con cada subtipo nuevo.

5.10 Agregación

Regla 12 · Agregación

La relación agregada se transforma con su regla habitual (aquí, una N:M → tabla BIBLIOTECA). Después, esa tabla se trata como si fuera una entidad: su **clave primaria** es la que viaja a las tablas de las relaciones en las que participa la agregación.

escribe es una 1:1 con `(1,1)` junto a la agregación y `(0,1)` junto a RESEÑA: toda reseña pertenece obligatoriamente a una posesión. Por la regla 7, se propaga la clave de la agregación (la PK de BIBLIOTECA) a RESENA como clave ajena `NOT NULL`. Es compuesta porque la PK de BIBLIOTECA es compuesta, y lleva `UNIQUE` para garantizar una sola reseña por posesión:

> **RESENA**(**id_resena**, nota, texto, fecha, recomendado, id_jugador NOT NULL, id_juego NOT NULL)
>
> *(id_jugador, id_juego) → BIBLIOTECA · UNIQUE (id_jugador, id_juego)*

Como la clave ajena apunta a BIBLIOTECA, **es imposible guardar la reseña de un juego que el jugador no tiene**: la regla del requisito 14 queda garantizada por la propia estructura.

5.11 El esquema relacional completo CE 6b CE 6c CE 6d CE 6e

Aplicando las doce reglas, el diagrama E/R de PixelVault se convierte en **20 tablas**:

> **USUARIO**(**id_usuario**, email UNIQUE, nombre, apellidos, fecha_registro)
>
> **JUGADOR**(***id_usuario → USUARIO***, nick UNIQUE, fecha_nacimiento, calle, numero, cp, ciudad, pais)
>
> **DESARROLLADOR**(***id_usuario → USUARIO***, cargo, *id_estudio → ESTUDIO* NOT NULL)
>
> **ESTUDIO**(**id_estudio**, nombre, pais, web)
>
> **VIDEOJUEGO**(**id_juego**, titulo, fecha_lanzamiento, pegi, tipo, *id_estudio → ESTUDIO* NOT NULL, *id_juego_original → VIDEOJUEGO* UNIQUE NULL)
>
> **IDIOMA_JUEGO**(***id_juego → VIDEOJUEGO***, **idioma**)
>
> **JUEGO_PAGO**(***id_juego → VIDEOJUEGO***, precio)
>
> **JUEGO_F2P**(***id_juego → VIDEOJUEGO***, tiene_micropagos)
>
> **GENERO**(**id_genero**, nombre UNIQUE)
>
> **JUEGO_GENERO**(***id_juego → VIDEOJUEGO***, ***id_genero → GENERO***)
>
> **PLATAFORMA**(**id_plataforma**, nombre, fabricante)
>
> **BIBLIOTECA**(***id_jugador → JUGADOR***, ***id_juego → VIDEOJUEGO***, fecha_adquisicion)
>
> **JUEGA**(***id_jugador → JUGADOR***, ***id_juego → VIDEOJUEGO***, ***id_plataforma → PLATAFORMA***, horas_jugadas)
>
> **AMISTAD**(***id_jugador → JUGADOR***, ***id_amigo → JUGADOR***)
>
> **LOGRO**(***id_juego → VIDEOJUEGO***, **num_logro**, nombre, descripcion, puntos)
>
> **DESBLOQUEA**(***id_jugador → JUGADOR***, **id_juego**, **num_logro**) — *(id_juego, num_logro) → LOGRO*
>
> **DLC**(**id_dlc**, titulo, precio, fecha_lanzamiento, *id_juego → VIDEOJUEGO* NOT NULL)
>
> **PEDIDO**(**num_pedido**, fecha, *id_jugador → JUGADOR* NOT NULL)
>
> **LINEA_PEDIDO**(***num_pedido → PEDIDO***, ***id_juego → JUEGO_PAGO***, precio_pagado)
>
> **RESENA**(**id_resena**, nota, texto, fecha, recomendado, id_jugador, id_juego) — *(id_jugador, id_juego) → BIBLIOTECA, UNIQUE, NOT NULL*

Y este es el mismo esquema dibujado como **diagrama relacional**, el tipo de figura que generan herramientas como Oracle SQL Data Modeler. Cada flecha va de la tabla que tiene la clave ajena a la tabla referenciada.

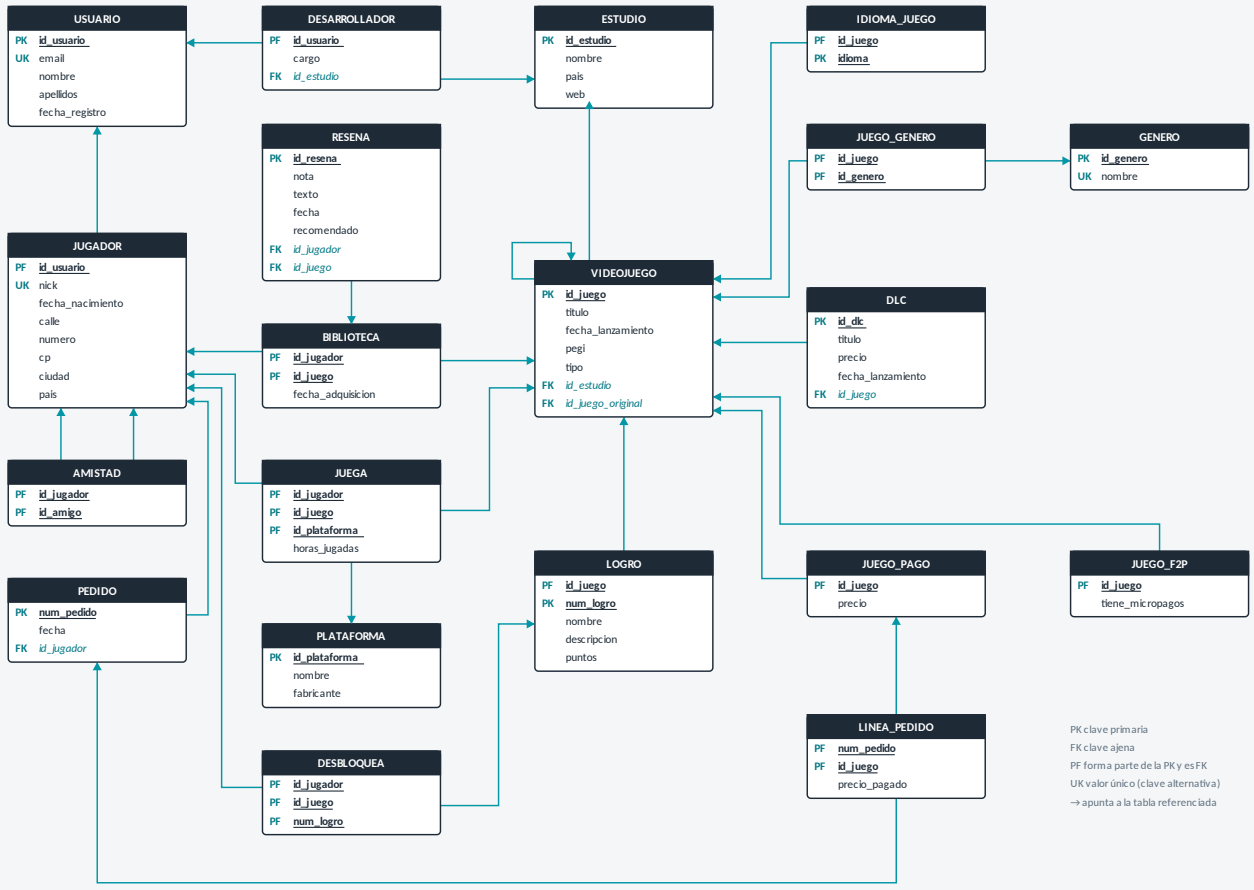


Diagrama relacional de PixelVault. PK: clave primaria · FK: clave ajena · PF: columna que es PK y FK a la vez · UK: única.
Abrir a tamaño completo.

5.12 Resumen de las reglas

Elemento del E/R	Se transforma en…
Entidad fuerte	Tabla; identificador → PK
Atributo compuesto	Una columna por componente
Atributo multivaluado	Tabla nueva: PK = PK de la entidad + valor
Atributo derivado	Nada (se calcula)
Relación 1:N	**Propagación de clave** del lado 1 al lado N; NOT NULL si el par del lado 1 tiene mínimo 1

Elemento del E/R	Se transforma en…
Relación 1:1	Propagación de clave con UNIQUE, desde la entidad que tiene el `(1,1)` junto a ella (o hacia la tabla con menos nulos)
Relación N:M	Tabla nueva: PK = las dos FK (+ lo que haga falta para no repetir)
Reflexiva	Igual que binaria, con columnas renombradas por rol
Ternaria	Tabla nueva con tres FK; PK según las cardinalidades
Débil por identificación	Tabla; PK = PK de la fuerte + discriminador; FK en cascada
Débil por existencia	Tabla; PK propia; FK NOT NULL en cascada
Jerarquía	Estrategia A, B o C según cobertura, solapamiento y relaciones propias
Agregación	La tabla de la relación agregada actúa como entidad

Resumen

- Cada **entidad** da una tabla; cada **N:M**, ternaria o multivaluado, una tabla nueva.
- En las **1:N** y **1:1** no se crea tabla: se resuelven por **propagación de clave**.
- En las 1:N la clave se **propaga** del lado 1 al lado N; el **mínimo del par del lado 1** decide si la clave ajena es `NOT NULL`, y el `UNIQUE` distingue una 1:1 de una 1:N.
- Las claves ajenas hacia **claves compuestas** llevan todas sus columnas.
- Las **jerarquías** tienen tres estrategias; en PixelVault usamos una tabla por entidad.

UD 2 · APARTADO 6

Reglas de integridad

Un esquema relacional no es solo una lista de tablas: también define qué datos son **válidos**. Las **reglas de integridad** son las condiciones que deben cumplir los datos en todo momento, y el SGBD se encarga de rechazar cualquier operación que las incumpla. En este apartado vemos cuáles son y decidimos cómo se comportan las claves ajenas de PixelVault cuando se borra algo.

6.1 Tipos de restricciones CE 6f

Tipo	Qué son	Ejemplos
Inherentes al modelo	Las impone el propio modelo relacional; no hay que declararlas	No puede haber dos filas iguales; cada celda es atómica
Semánticas (de usuario)	Las define el diseñador a partir de los requisitos	Clave primaria, clave ajena, `NOT NULL`, `UNIQUE`, `CHECK`

Las restricciones semánticas se agrupan en tres reglas: integridad de **entidad**, integridad **referencial** e integridad de **dominio**.

6.2 Integridad de entidad

Regla de integridad de entidad

Ningún atributo que forme parte de la **clave primaria** puede tomar el valor **NULL**, y el valor de la clave primaria no puede repetirse.

La razón es sencilla: la clave primaria sirve para identificar cada fila. Una fila con clave nula sería una fila que no se puede identificar ni referenciar.

En LOGRO, la clave primaria es `(id_juego, num_logro)`: **ninguna de las dos** columnas puede ser nula. No basta con que una de ellas tenga valor.

6.3 Integridad referencial

Regla de integridad referencial

Si una tabla tiene una **clave ajena**, cada valor de esa clave ajena debe **coincidir con un valor existente** de la clave primaria referenciada, o bien ser **NULL** (si la columna lo admite).

Dicho de otra forma: no puede haber referencias «huérfanas». Veamos qué operaciones podrían romperla en PixelVault:

Operación	¿Qué pasaría?	¿Rompe la integridad?
Insertar en VIDEOJUEGO un juego con `id_estudio = 99`, que no existe	El juego apuntaría a un estudio inexistente	**Sí**: el SGBD la rechaza
Insertar en BIBLIOTECA la fila (101, 6) cuando el jugador 101 y el juego 6 existen	Correcto	No
Borrar de ESTUDIO Supergiant Games (15), que tiene juegos	Hades quedaría apuntando a un estudio borrado	**Sí**, salvo que se defina otra acción
Cambiar el `id_juego` de Hades de 6 a 60	Las filas de BIBLIOTECA, LOGRO… que dicen 6 quedarían huérfanas	**Sí**

Las inserciones incorrectas simplemente se rechazan. Pero ante un **borrado** (o una modificación de la clave primaria) en la tabla referenciada, el diseñador puede elegir qué debe hacer el SGBD.

6.4 ¿Qué pasa al borrar? Acciones referenciales

Acción	Qué hace al borrar la fila referenciada	Ejemplo en PixelVault
Restringir (*NO ACTION / RESTRICT*)	Impide el borrado si hay filas que la referencian. Es el comportamiento por defecto.	No se puede borrar un estudio que tiene juegos.
Cascada (*CASCADE*)	Borra también las filas que la referencian.	Al borrar un juego se borran sus logros y sus DLC.
Poner a nulo (*SET NULL*)	Pone a NULL la clave ajena de las filas que la referencian. Solo si la FK admite nulos.	Si se borra Hades, Hades II deja de tener original.
Valor por defecto (*SET DEFAULT*)	Pone la clave ajena a su valor por defecto.	Poco habitual.

Lo que admite Oracle

Oracle implementa las tres primeras para el borrado: la restrictiva (por defecto, si no se indica nada), `ON DELETE CASCADE` y `ON DELETE SET NULL`. **No** tiene `ON UPDATE CASCADE`: en Oracle la clave primaria no se modifica, y esa es una razón más para usar claves subrogadas que no cambian nunca. `SET DEFAULT` tampoco está disponible.

Así se declara una clave ajena con borrado en cascada en Oracle. Lo estudiarás a fondo en la UD 5; aquí solo importa ver dónde queda reflejada la decisión de diseño:

Clave ajena de LOGRO hacia VIDEOJUEGO

```
-- Si se borra un videojuego, se borran automáticamente sus logros
CONSTRAINT fk_logro_juego
    FOREIGN KEY (id_juego)
```

```
    REFERENCES videojuego (id_juego)
    ON DELETE CASCADE
```

Decisiones para PixelVault

Para cada clave ajena hay que decidir la acción mirando **el significado** de los datos. Esta es nuestra propuesta:

Tabla (FK)	Referencia a	Acción al borrar	Justificación
JUGADOR, DESARROLLADOR	USUARIO	CASCADE	Son la misma persona: sin usuario no hay perfil.
DESARROLLADOR	ESTUDIO	Restringir	No se borra un estudio con desarrolladores dados de alta.
VIDEOJUEGO	ESTUDIO	Restringir	No se borra un estudio del que se venden juegos.
VIDEOJUEGO (`id_juego_original`)	VIDEOJUEGO	SET NULL	Si desaparece el original, la secuela sigue existiendo.
IDIOMA_JUEGO, JUEGO_PAGO, JUEGO_F2P, JUEGO_GENERO	VIDEOJUEGO	CASCADE	Son datos del propio juego.
JUEGO_GENERO	GENERO	Restringir	No se borra un género en uso.
LOGRO, DLC	VIDEOJUEGO	CASCADE	Entidades débiles: no existen sin su juego (requisito 13).
BIBLIOTECA, JUEGA	VIDEOJUEGO	Restringir	No se retira un juego que algún jugador tiene o ha jugado.
BIBLIOTECA, JUEGA, AMISTAD, DESBLOQUEA	JUGADOR	CASCADE	Son datos del jugador.
JUEGA	PLATAFORMA	Restringir	Hay estadísticas que dependen de la plataforma.
DESBLOQUEA	LOGRO	CASCADE	Si el logro desaparece, sus desbloqueos también.
RESENA	BIBLIOTECA	CASCADE	Sin posesión no puede haber reseña.
PEDIDO	JUGADOR	Restringir	Los pedidos se conservan por obligación contable.
LINEA_PEDIDO	PEDIDO	CASCADE	Las líneas son parte del pedido.
LINEA_PEDIDO	JUEGO_PAGO	Restringir	No se borra un juego que figura en ventas.

Bajas lógicas

Fíjate en una consecuencia: como PEDIDO restringe el borrado, un jugador con compras **no se puede borrar**. En la práctica, las empresas no borran estos registros, sino que hacen una **baja lógica**: añaden una columna `activo` (o `fecha_baja`) y la marcan. Así se conservan los pedidos y se respeta la integridad.

Cuidado con las cascadas encadenadas

Las cascadas se propagan: borrar un USUARIO borra su fila de JUGADOR, que borra su BIBLIOTECA, que borra sus RESENA... Es cómodo, pero un borrado puede llevarse por delante muchos datos. Usa CASCADE solo cuando las filas hijas **no tengan sentido** sin la madre.

6.5 Integridad de dominio

Además de las claves, cada columna puede tener restricciones sobre sus valores. Todas se pueden expresar en el esquema y, después, en Oracle:

Restricción	Significado	Ejemplo en PixelVault
Tipo de dato	El valor pertenece al tipo de la columna	`fecha_lanzamiento` es una fecha
`NOT NULL`	Valor obligatorio	`email`, `titulo`, `fecha` del pedido
`UNIQUE`	Valor no repetido (clave alternativa)	`nick`, `email`, nombre de género
`CHECK`	Condición sobre los valores de **la misma fila**	`pegi IN (3, 7, 12, 16, 18)`; `nota BETWEEN 1 AND 10`; `precio >= 0`; `tipo IN ('PAGO','F2P')`
`DEFAULT`	Valor si no se indica otro	`fecha_registro` = fecha actual

El límite del CHECK

Un `CHECK` solo puede mirar los valores **de la propia fila**. «La nota está entre 1 y 10» es un CHECK. Pero «para reseñar hay que haber jugado 2 horas» necesita consultar **otra tabla** (JUEGA), y eso ya no lo puede hacer un CHECK. Esas reglas son las **restricciones no representables** del apartado 8.

6.6 Resumen: del diseño a Oracle

Regla de integridad	En el esquema	En Oracle (UD 5)
De entidad	Clave primaria subrayada	`PRIMARY KEY`

Regla de integridad	En el esquema	En Oracle (UD 5)
Referencial	Clave ajena con flecha	`FOREIGN KEY ... REFERENCES ... [ON DELETE ...]`
Clave alternativa	`UNIQUE`	`UNIQUE`
Obligatoriedad	`NOT NULL`	`NOT NULL`
De dominio	Anotación de valores válidos	`CHECK`, tipo de dato, `DEFAULT`

Resumen

- **Integridad de entidad**: la clave primaria no puede ser nula ni repetirse.
- **Integridad referencial**: una clave ajena vale lo mismo que una clave primaria existente, o es nula.
- Ante un borrado se puede **restringir**, borrar en **cascada** o **poner a nulo**; Oracle no tiene `ON UPDATE CASCADE`.
- La acción de cada clave ajena se decide según el **significado** de los datos.
- Las reglas sobre valores de una fila se expresan con `NOT NULL`, `UNIQUE` y `CHECK`; las que implican otras tablas, no.

UD 2 · APARTADO 7

Normalización

En el apartado 1 vimos que guardar las ventas de PixelVault en una sola tabla provocaba redundancia y anomalías. La **normalización** es el procedimiento formal para detectar y eliminar esos problemas: se comprueba si cada tabla cumple una serie de condiciones, las **formas normales**, y si no las cumple se **descompone** en tablas más pequeñas sin perder información.

La normalización sirve para dos cosas:

- **Comprobar** que el esquema obtenido del diagrama E/R es correcto.
- **Diseñar a partir de documentos existentes** (tickets, facturas, hojas de cálculo) cuando no hay un diagrama E/R previo.

7.1 Dependencias funcionales CE 6g

Todas las formas normales se definen a partir de un concepto: la dependencia funcional.

Dependencia funcional (DF)

> Se dice que **Y depende funcionalmente de X**, y se escribe **X → Y**, si cada valor de X determina **un único valor** de Y. A X se le llama **determinante**.

Ejemplos en PixelVault:

- `id_juego → titulo` : conocido el identificador del juego, su título es uno solo.
- `nick → email` : cada *nick* tiene un único correo.
- `titulo → id_juego` **no** tiene por qué cumplirse: podría haber dos juegos con el mismo título (dos *remakes*, por ejemplo).

> **Las DF salen de los requisitos, no de los datos**
>
> Que en las filas de ejemplo no se repita ningún título no significa que `titulo → id_juego`. Las dependencias se deducen de las **reglas del negocio**, no de los datos que haya en un momento dado.

Hay tres tipos de dependencia que vamos a necesitar:

Dependencia completa

> Y depende de **todo** X y no de una parte. En la línea de un pedido, `(num_pedido, id_juego) → precio_pagado` es completa: el precio depende del pedido **y** del juego.

Dependencia parcial

> Y depende solo de **una parte** de una clave compuesta. `(num_pedido, id_juego) → titulo` es parcial, porque basta `id_juego → titulo`.

Dependencia transitiva

> Y depende de X **a través** de otro atributo Z que no es clave: X → Z y Z → Y. Por ejemplo, `num_pedido → nick` y `nick → email`, luego `email` depende transitivamente de `num_pedido`.

7.2 El punto de partida: el ticket de PixelVault

El departamento de ventas nos pasa un ticket de compra y nos pide una base de datos que pueda generarlo:

Pedido n.º	5001
Fecha	2026-09-01
Jugador	NoctisRJ · noctis@ejemplo.es · España (IVA 21 %)
Juego 1	6 · Hades · Supergiant Games (15) · EE. UU. · 24,50 €
Juego 2	5 · Hollow Knight · Team Cherry (16) · Australia · 14,79 €
Total	39,29 €

Si pasamos varios tickets a una tabla tal cual, obtenemos una tabla **sin normalizar**, con un **grupo repetitivo** (los juegos del pedido) dentro de cada fila:

num_pedido	fecha	nick	email	pais	iva	juegos (id, título, estudio, país estudio, precio)	total
5001	2026-09-01	NoctisRJ	noctis@ejemplo.es	España	21	(6, Hades, 15 Supergiant Games, EE. UU., 24,50); (5, Hollow Knight, 16 Team Cherry, Australia, 14,79)	39,29
5002	2026-09-02	LaTarasca	tarasca@ejemplo.es	España	21	(6, Hades, 15 Supergiant Games, EE. UU., 19,99); (8, Stardew Valley, 17 ConcernedApe, EE. UU., 13,99)	33,98

Primero quitamos lo que no se debe guardar: `total` es un **atributo derivado** (se calcula sumando los precios), así que desaparece.

7.3 Primera forma normal (1FN)

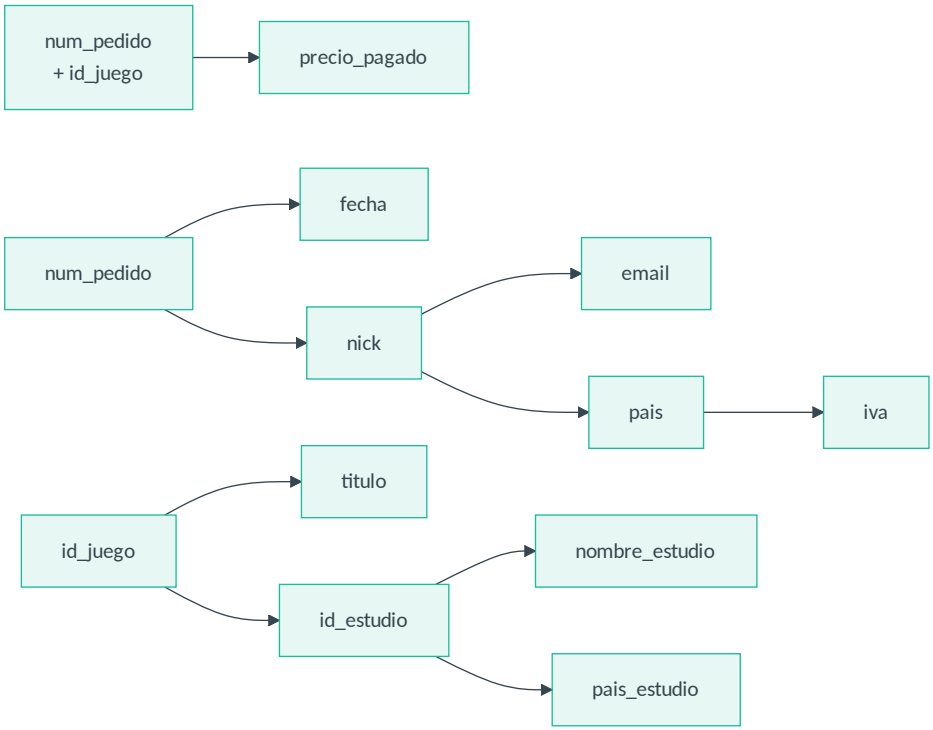
1FN

Una tabla está en primera forma normal si **todos sus atributos son atómicos**: cada celda contiene un único valor y no hay grupos repetitivos.

Nuestra tabla no está en 1FN: la columna de juegos contiene una lista. Para arreglarlo, se crea **una fila por cada juego** del pedido, repitiendo los datos del pedido, y se busca la nueva clave primaria. Como un pedido tiene varios juegos, `num_pedido` ya no basta: la clave es **(num_pedido, id_juego)**.

num_pedido	id_juego	fecha	nick	email	pais	iva	titulo	id_estudio	nombre_es
5001	6	2026-09-01	NoctisRJ	noctis@ejemplo.es	España	21	Hades	15	Supergia Games
5001	5	2026-09-01	NoctisRJ	noctis@ejemplo.es	España	21	Hollow Knight	16	Team Che
5002	6	2026-09-02	LaTarasca	tarasca@ejemplo.es	España	21	Hades	15	Supergia Games
5002	8	2026-09-02	LaTarasca	tarasca@ejemplo.es	España	21	Stardew Valley	17	Concerne

Ya está en 1FN, pero la redundancia es enorme: los datos de Hades y de Supergiant Games se repiten en cada pedido que los incluye. Para seguir necesitamos las dependencias funcionales de la tabla:



¿Por qué `precio_pagado` depende de los dos?

Porque Hades se vendió a 24,50 € en el pedido 5001 y a 19,99 € (en oferta) en el 5002. El precio pagado no lo determina solo el juego ni solo el pedido.

7.4 Segunda forma normal (2FN)

2FN

Una tabla está en segunda forma normal si está en 1FN y **todo atributo que no forma parte de la clave depende de la clave completa**, no de una parte de ella. Es decir, no hay **dependencias parciales**.

Atajo

Si la clave primaria es **simple** (una sola columna), la tabla está automáticamente en 2FN si está en 1FN: no puede haber dependencias de «una parte» de la clave.

En nuestra tabla hay dependencias parciales:

- De `num_pedido` solo: `fecha`, `nick`, `email`, `pais`, `iva`.
- De `id_juego` solo: `titulo`, `id_estudio`, `nombre_estudio`, `pais_estudio`.
- De la clave completa: solo `precio_pagado`.

Para pasar a 2FN, cada grupo de atributos se lleva a una tabla con **la parte de la clave de la que depende**, que será su clave primaria:

```
PEDIDO(num_pedido, fecha, nick, email, pais, iva)
JUEGO(id_juego, titulo, id_estudio, nombre_estudio, pais_estudio)
LINEA_PEDIDO(num_pedido → PEDIDO, id_juego → JUEGO, precio_pagado)
```

Hades ya solo aparece una vez en JUEGO. Pero aún quedan problemas: en PEDIDO, el correo de NoctisRJ se repite en todos sus pedidos.

7.5 Tercera forma normal (3FN)

3FN

Una tabla está en tercera forma normal si está en 2FN y **ningún atributo que no es clave depende de otro atributo que no es clave**. Es decir, no hay **dependencias transitivas**.

Una forma fácil de recordarlo: en 3FN, cada atributo no clave depende de «**la clave, toda la clave y nada más que la clave**».

Buscamos dependencias entre atributos no clave:

- En PEDIDO: `nick → email, pais` y `pais → iva`. Son transitivas.
- En JUEGO: `id_estudio → nombre_estudio, pais_estudio`. Es transitiva.
- En LINEA_PEDIDO: solo hay un atributo no clave. Ya está en 3FN.

Para pasar a 3FN, se saca cada dependencia transitiva a una tabla nueva cuya clave es **el determinante**, que se queda en la tabla original como **clave ajena**:

```
PAIS(pais, iva)
JUGADOR(nick, email, pais → PAIS)
PEDIDO(num_pedido, fecha, nick → JUGADOR)
ESTUDIO(id_estudio, nombre_estudio, pais_estudio)
JUEGO(id_juego, titulo, id_estudio → ESTUDIO)
LINEA_PEDIDO(num_pedido → PEDIDO, id_juego → JUEGO, precio_pagado)
```

Comprobemos que han desaparecido las anomalías del apartado 1:

Anomalía	Antes	Después
Modificación	Cambiar el correo de NoctisRJ obligaba a tocar muchas filas	Se cambia en **una** fila de JUGADOR
Inserción	No se podía dar de alta un estudio sin ventas	Se inserta en ESTUDIO sin más
Borrado	Anular el único pedido de FromSoftware borraba el estudio	Se borra el pedido; el estudio sigue en ESTUDIO

Compara con el diseño E/R

Estas tablas son prácticamente las mismas que obtuvimos en el apartado 5 a partir del diagrama: PEDIDO, JUGADOR, VIDEOJUEGO, ESTUDIO y LINEA_PEDIDO. **Cuando el diagrama E/R está bien hecho, el esquema sale ya normalizado.** La normalización ha descubierto además una tabla nueva, PAIS, porque el ticket incluía el IVA, un dato que no estaba en los requisitos del cliente. Es un buen ejemplo de por qué conviene revisar los documentos reales de la empresa.

7.6 Forma normal de Boyce-Codd (FNBC)

La 3FN resuelve casi todos los casos reales, pero hay situaciones raras (con varias claves candidatas compuestas que se solapan) en las que una tabla en 3FN todavía tiene redundancia. Para ellas existe una forma normal más estricta.

FNBC

Una tabla está en forma normal de Boyce-Codd si **todo determinante es clave candidata**. Es decir, para cada dependencia X → Y, X identifica las filas de la tabla.

Ejemplo: los entrenadores de PixelVault

PixelVault lanza un servicio de entrenamiento para sus jugadores más competitivos, con estas reglas:

- Cada **entrenador** está especializado en **un único videojuego**.
- Cada jugador tiene, **para cada videojuego**, **un único entrenador**.
- Un videojuego puede tener varios entrenadores.

id_jugador	videojuego	entrenador
101	Counter-Strike 2	Marta
102	Counter-Strike 2	Marta
104	Counter-Strike 2	Íker
101	Fortnite	Lucas

Las dependencias funcionales son:

- `(id_jugador, videojuego) → entrenador` (un entrenador por jugador y juego).
- `entrenador → videojuego` (cada entrenador, un solo juego).

Claves candidatas: `(id_jugador, videojuego)` y también `(id_jugador, entrenador)`, porque conociendo el entrenador se sabe el juego.

- ¿Está en **3FN**? Sí: el único atributo que depende de algo que no es clave es `videojuego`, pero `videojuego` **forma parte de una clave candidata**, y la 3FN lo permite.
- ¿Está en **FNBC**? **No**: `entrenador` es un determinante (`entrenador → videojuego`) y **no** es clave candidata.

El problema se nota en los datos: «Marta entrena Counter-Strike 2» está repetido, y si el jugador 104 deja el servicio y era el único alumno de Íker, perdemos que Íker entrena Counter-Strike 2 (anomalía de borrado).

La solución es descomponer usando la dependencia que molesta:

> **ENTRENADOR**(**entrenador**, videojuego)
> **ENTRENAMIENTO**(**id_jugador**, ***entrenador → ENTRENADOR***)

Un precio a pagar

Con esta descomposición, la regla «un jugador tiene un solo entrenador por juego» ya no se puede garantizar con una clave (el juego no está en ENTRENAMIENTO). Pasar a FNBC a veces obliga a **perder una dependencia**, que habrá que controlar de otra forma y documentar como restricción. Por eso, en la práctica, muchos diseños se quedan en 3FN.

Ampliación: más allá de la FNBC

Existen la **cuarta forma normal** (4FN), que trata las *dependencias multivaluadas*, y la **quinta** (5FN), que trata las *dependencias de combinación*. Un ejemplo de problema de 4FN: guardar en una misma tabla (jugador, plataforma, idioma_preferido), cuando las plataformas y los idiomas de un jugador son independientes entre sí; habría que repetir cada plataforma con cada idioma. Si has hecho bien el diagrama E/R (plataformas e idiomas como relaciones o multivaluados separados), no te encontrarás con este caso.

7.7 Desnormalización

A veces, por **rendimiento**, se decide conscientemente dejar una tabla sin normalizar del todo: por ejemplo, guardar `nota_media` en VIDEOJUEGO para no recalcularla en cada visita a la ficha del juego. Es la **desnormalización**. Es una decisión del diseño físico, se toma después de normalizar (nunca en lugar de normalizar), y obliga a mantener el dato copiado coherente (en Oracle, con disparadores, que verás en la UD 6).

7.8 Procedimiento paso a paso

1. Elimina los **atributos derivados**.
2. Identifica la **clave primaria** de la tabla de partida.
3. **1FN**: elimina los grupos repetitivos (una fila por valor) y revisa la clave.
4. Escribe todas las **dependencias funcionales**.
5. **2FN**: si la clave es compuesta, saca a otra tabla los atributos que dependen de solo una parte.
6. **3FN**: saca a otra tabla los atributos que dependen de un atributo no clave; el determinante queda como clave ajena.
7. **FNBC**: comprueba que todo determinante es clave candidata.
8. Comprueba que puedes **reconstruir** la información original uniendo las tablas por sus claves.

Resumen

- Una **dependencia funcional** X → Y dice que X determina un único valor de Y. Se deducen de las reglas del negocio.
- **1FN**: valores atómicos. **2FN**: sin dependencias parciales. **3FN**: sin dependencias transitivas. **FNBC**: todo determinante es clave candidata.
- Cada paso se resuelve **descomponiendo** la tabla; el determinante queda como clave ajena.
- Un buen diagrama E/R produce tablas ya normalizadas; la normalización lo **comprueba**.

UD 2 · APARTADO 8

Restricciones no representables

Por muy completo que sea, un diagrama E/R no puede expresar todas las reglas del negocio. Y el esquema relacional, con sus claves primarias y ajenas, tampoco. Las reglas que se quedan fuera no desaparecen: hay que **analizarlas y documentarlas** para que se implementen después en Oracle (con restricciones `CHECK` o con disparadores) o en la aplicación. Si no se documentan, nadie las implementará y la base de datos aceptará datos incorrectos.

8.1 ¿Qué no se puede representar? CE 6h

El diagrama E/R expresa entidades, atributos, relaciones, cardinalidades, dependencias y jerarquías. Todo lo demás queda fuera. Las situaciones típicas son:

Tipo de restricción	Ejemplo en PixelVault	¿Por qué no cabe en el diagrama?
De dominio (valores válidos)	La nota de una reseña va de 1 a 10	El diagrama no muestra dominios
Entre atributos de la misma fila	Un juego no puede ser secuela de sí mismo	Las cardinalidades no comparan valores
Entre tablas	Para reseñar hay que haber jugado 2 horas	Implica datos de otra relación (*juega*)
Temporales	Una secuela no puede salir antes que el original	Compara fechas de filas distintas
Mínimos en N:M	Todo pedido tiene al menos un juego `(1,N)`	Las claves no pueden obligar a que existan filas en otra tabla
Cobertura de jerarquías	Todo juego es de pago o free-to-play, y solo uno	Las tablas de subtipos no obligan a existir

Una pista para encontrarlas

Relee los requisitos buscando palabras como **«solo si»**, **«al menos»**, **«como máximo»**, **«no puede»**, **«antes de»**, **«después de»**, **«entre»**. Casi siempre esconden una restricción. Después repasa todas las cardinalidades con mínimo 1 en relaciones N:M y todas las jerarquías totales o exclusivas.

8.2 El caso del mínimo en las relaciones N:M

Este es el más fácil de pasar por alto. En la relación *incluye* aparece `(1,N)` junto a DE_PAGO: todo pedido incluye **al menos un** juego. Pero al pasar a tablas:

```
PEDIDO(num_pedido, fecha, id_jugador → JUGADOR NOT NULL)
LINEA_PEDIDO(num_pedido → PEDIDO, id_juego → JUEGO_PAGO, precio_pagado)
```

Nada impide insertar un pedido y no insertar ninguna línea. La clave ajena garantiza que cada línea tenga pedido, **no** que cada pedido tenga líneas. Ese mínimo 1 se ha perdido en la transformación y hay que documentarlo. Lo mismo ocurre con el `(1,N)` junto a GÉNERO en *pertenece*: todo juego debe tener al menos un género.

8.3 Catálogo de restricciones de PixelVault

Este es el análisis completo para nuestro caso. La última columna indica cómo se implementará en Oracle y en qué unidad lo aprenderás.

Código	Restricción	Origen	Tablas	Implementación prevista
R-01	`pegi` solo admite 3, 7, 12, 16 o 18	Req. 2	VIDEOJUEGO	`CHECK` (UD 5)
R-02	`nota` entre 1 y 10	Req. 14	RESENA	`CHECK` (UD 5)
R-03	`precio`, `precio_pagado` y `horas_jugadas` no negativos; `puntos` mayor que 0	Sentido común	JUEGO_PAGO, DLC, LINEA_PEDIDO, JUEGA, LOGRO	`CHECK` (UD 5)
R-04	`tipo` solo admite `'PAGO'` o `'F2P'`	Req. 11	VIDEOJUEGO	`CHECK` (UD 5)
R-05	Un jugador no puede ser amigo de sí mismo	Req. 18	AMISTAD	`CHECK (id_jugador <> id_amigo)` (UD 5)
R-06	La amistad se guarda una sola vez: si existe (101, 102) no se guarda (102, 101)	Decisión de diseño	AMISTAD	`CHECK (id_jugador < id_amigo)` (UD 5)
R-07	Un juego no puede ser secuela de sí mismo	Req. 17	VIDEOJUEGO	`CHECK (id_juego_original <> id_juego)` (UD 5)
R-08	La secuela no puede lanzarse antes que el original	Req. 17	VIDEOJUEGO	Disparador (UD 6)
R-09	Un DLC no puede lanzarse antes que su juego	Sentido común	DLC, VIDEOJUEGO	Disparador (UD 6)

Código	Restricción	Origen	Tablas	Implementación prevista
R-10	Todo juego tiene fila en JUEGO_PAGO o en JUEGO_F2P según su `tipo`, y solo en esa	Req. 11 (jerarquía total y exclusiva)	VIDEOJUEGO, JUEGO_PAGO, JUEGO_F2P	Disparadores o procedimiento de alta (UD 6)
R-11	Todo pedido tiene al menos una línea	Req. 9, cardinalidad (1,N)	PEDIDO, LINEA_PEDIDO	Procedimiento de alta de pedidos (UD 6)
R-12	Todo juego pertenece al menos a un género	Req. 4, cardinalidad (1,N)	VIDEOJUEGO, JUEGO_GENERO	Procedimiento de alta de juegos (UD 6)
R-13	Para reseñar un juego hay que haber jugado al menos 2 horas en total	Req. 15	RESENA, JUEGA	Disparador (UD 6)
R-14	No se vende un juego a un jugador cuya edad sea inferior al PEGI	Req. 16	LINEA_PEDIDO, PEDIDO, JUGADOR, VIDEOJUEGO	Disparador (UD 6)
R-15	Solo se desbloquean logros de juegos que el jugador tiene en su biblioteca	Sentido común	DESBLOQUEA, BIBLIOTECA	Disparador (UD 6)
R-16	Un DLC solo se compra si se tiene el juego base	Req. 19	—	**Requisito incompleto**: ver más abajo

Fíjate en que R-05, R-06 y R-07 **no se pueden dibujar** en el diagrama, pero sí se pueden implementar con un `CHECK` porque solo comparan columnas de **la misma fila**. En cambio, R-08 compara la fecha de una fila con la de **otra** fila (la del juego original), y eso ya no lo puede hacer un `CHECK`.

> ⚠ **R-16: cuando una restricción descubre un fallo en los requisitos**
>
> Al analizar el requisito 19 descubrimos algo importante: **nuestro modelo no guarda la compra de DLC**. La relación *incluye* solo conecta pedidos con juegos de pago. No se puede comprobar una regla sobre algo que no se registra. Lo correcto no es inventarse la solución, sino **documentarlo y preguntar al cliente**: ¿los DLC se venden en los pedidos? Si la respuesta es sí, habrá que volver al diagrama E/R y añadir una relación entre PEDIDO y DLC. Este «volver atrás» es el carácter iterativo del diseño del que hablábamos en el apartado 1.

8.4 Cómo documentar una restricción

Además del catálogo, cada restricción importante se describe en una **ficha** con un formato fijo, para que quien la implemente no tenga dudas. Esta es la ficha de R-13:

Ficha de restricción R-13

Código: R-13

Nombre: Mínimo de horas para reseñar.

Descripción: un jugador solo puede escribir la reseña de un videojuego si la suma de sus `horas_jugadas` en ese juego, en todas las plataformas, es de al menos 2.

Origen: requisito 15 del cliente.

Tablas implicadas: RESENA (se comprueba al insertar), JUEGA (se consulta).

Operaciones que pueden incumplirla: insertar en RESENA. *(Modificar JUEGA para bajar las horas después de reseñar se considera imposible: las horas solo crecen.)*

Acción si se incumple: rechazar la inserción con el mensaje «Necesitas haber jugado al menos 2 horas para reseñar este juego».

Implementación prevista: disparador `BEFORE INSERT` sobre RESENA (UD 6).

Ejemplo: NoctisRJ tiene 1,5 h en Elden Ring (PC) y 0,4 h (PS5). Total: 1,9 h. Su reseña se rechaza.

8.5 ¿Dónde se implementan?

Mecanismo	Qué puede comprobar	Cuándo lo verás
`NOT NULL`, `UNIQUE`, `CHECK`	Condiciones sobre los valores de una sola fila	UD 5
Disparadores (*triggers*)	Cualquier condición, consultando otras tablas	UD 6
Procedimientos almacenados	Operaciones completas que respetan las reglas (por ejemplo, «dar de alta un pedido con sus líneas»)	UD 6
La aplicación	Validaciones para dar mensajes amigables al usuario	Otros módulos

No confíes solo en la aplicación

Es tentador dejar todas estas comprobaciones al programa que usa la base de datos. Pero a una base de datos se accede desde muchos sitios: la web, la app móvil, un script de carga, SQL Developer... Si la regla solo está en uno de ellos, los demás se la saltarán. Las reglas importantes deben estar **en la base de datos**.

Resumen

- Las **restricciones no representables** son reglas del negocio que ni el diagrama E/R ni las claves pueden expresar.
- Típicas: dominios de valores, comparaciones entre columnas o entre filas, reglas que consultan otras tablas, **mínimos 1 en N:M** y **cobertura de jerarquías**.

- Se recogen en un **catálogo** y, las importantes, en una **ficha** con descripción, origen, tablas, acción e implementación prevista.
- Las de una sola fila se implementarán con **CHECK**; las demás, con **disparadores** o procedimientos.
- Analizar restricciones a veces revela **requisitos incompletos**: hay que preguntar al cliente y volver al diseño.

Oracle SQL Data Modeler

Hasta ahora hemos dibujado los diagramas a mano o con una herramienta de dibujo genérica. En la empresa se usan **herramientas CASE** de modelado, que además de dibujar **entienden** el modelo: comprueban que es coherente, lo transforman en tablas y generan el script SQL. La de Oracle es **Oracle SQL Developer Data Modeler**, gratuita. En este apartado aprenderás a usarla con una parte de PixelVault.

9.1 Qué es y qué hace CE 6a

Oracle SQL Developer Data Modeler (la llamaremos simplemente *Data Modeler*) permite trabajar con los tres niveles del diseño en un mismo **proyecto de diseño** (archivo `.dmd`):

Modelo en Data Modeler	Equivale a...	Lo que haces en él
Logical (modelo lógico)	Nuestro diagrama E/R	Entidades, atributos, relaciones, subtipos
Relational (modelo relacional)	Nuestro esquema relacional	Tablas, columnas, claves primarias y ajenas
Physical (modelo físico)	El diseño físico en Oracle	*Tablespaces*, almacenamiento, detalles del gestor

El flujo de trabajo es exactamente el de la unidad:



Se puede usar como programa independiente o desde **Oracle SQL Developer** (menú *View → Data Modeler*). También hace **ingeniería inversa**: conectado a una base de datos existente, dibuja su diagrama.

9.2 Otra notación: pata de gallo y Barker

Data Modeler **no usa la notación Chen**. En el modelo lógico ofrece las notaciones **Barker**, **Bachman** e **Ingeniería de la Información** (pata de gallo); se cambia desde el menú contextual del diagrama, en la opción de notación. Los conceptos son los mismos que ya conoces; solo cambia el dibujo:

Concepto	Chen (estos apuntes)	Data Modeler
Entidad	Rectángulo	Rectángulo con los atributos **dentro**
Atributo	Elipse	Una línea dentro del rectángulo
Identificador	Subrayado	Marcado como *Primary UID* (en Barker, con `#`)

Concepto	Chen (estos apuntes)	Data Modeler
Relación	Rombo con nombre	**Línea** entre las entidades, sin rombo
Máximo N	`N` junto a la entidad	**Pata de gallo** (tres rayitas) en el extremo
Mínimo 0 (opcional)	`0` junto a la entidad	Línea discontinua (Barker) o círculo (pata de gallo)
Entidad débil por identificación	Doble rectángulo y doble rombo	Relación marcada como **Identifying**
Jerarquía	Triángulo ES UN	Subtipos dibujados **dentro** del supertipo (Barker)

Buena noticia: la pata de gallo usa nuestro mismo convenio

En pata de gallo, el símbolo de «muchos» y el círculo de «opcional» se dibujan junto a la entidad que se cuenta, igual que nuestros pares del **extremo opuesto**. En *desarrolla*, la pata de gallo y el círculo aparecen junto a VIDEOJUEGO, justo donde nosotros escribimos `(0,N)`.

Tres limitaciones importantes

El modelo lógico de Data Modeler solo tiene relaciones **binarias** y **sin atributos**. Por eso, al pasar nuestro diagrama Chen a la herramienta:

1. Una **N:M con atributos** (*posee* con `fecha_adquisicion`) se modela como una **entidad** (BIBLIOTECA) con dos relaciones 1:N.
2. Una **ternaria** (*juega*) se modela como una **entidad** (JUEGA) con tres relaciones 1:N.
3. Una **agregación** se modela igual: la relación agregada se convierte en entidad y de ella cuelga la otra relación.

Es lo mismo que hicimos al pasar a tablas en el apartado 5, solo que un paso antes.

9.3 Práctica guiada: una parte de PixelVault

Vamos a modelar este fragmento del caso: **ESTUDIO**, **VIDEOJUEGO**, **GENERO** (con la N:M *pertenece*) y **LOGRO** (débil por identificación). Al final generarás el script DDL.

Nombres de menús y botones

La herramienta está en inglés. Los nombres de las opciones pueden variar ligeramente de una versión a otra; si no encuentras un botón, pasa el ratón por encima de los iconos de la barra de herramientas del diagrama para ver su nombre.

Paso 1 · Crear y guardar el diseño

1. Abre Data Modeler. Al arrancar se crea un diseño vacío con un modelo lógico (*Logical*) y un modelo relacional (*Relational_1*) en el navegador de la izquierda.

2. Guárdalo desde **File → Save As** con el nombre `pixelvault`. Se crea el archivo `pixelvault.dmd` y una carpeta con el mismo nombre. **Entrega siempre las dos cosas**: sin la carpeta, el diseño no se abre.
3. Abre el diagrama del modelo lógico (doble clic en *Logical*).

Paso 2 · Crear las entidades

1. Pulsa el botón **New Entity** de la barra de herramientas y haz clic en el lienzo. Se abre el cuadro de propiedades de la entidad.
2. En **General**, escribe el nombre: `ESTUDIO`.
3. En **Attributes**, añade con el botón **+** los atributos `id_estudio`, `nombre`, `pais` y `web`. Para cada uno elige un tipo lógico (por ejemplo, `NUMERIC` para el identificador y `VARCHAR` con su tamaño para los textos).
4. Marca `id_estudio` como **Primary UID**: es el identificador (nuestra clave subrayada).
5. Marca como obligatorios (*Mandatory*) los atributos que no puedan ser nulos: `nombre`.
6. Repite el proceso para `VIDEOJUEGO` (`id_juego`, `titulo`, `fecha_lanzamiento`, `pegi`), `GENERO` (`id_genero`, `nombre`) y `LOGRO` (`num_logro`, `nombre`, `descripcion`, `puntos`).

> ⚠ **No añadas las claves ajenas**
>
> En el modelo lógico **no** se escribe `id_estudio` dentro de VIDEOJUEGO. Las claves ajenas las crea la herramienta al transformar las relaciones, igual que hacíamos nosotros en el apartado 5. Si las añades a mano, saldrán duplicadas.

Paso 3 · Crear las relaciones

1. **ESTUDIO desarrolla VIDEOJUEGO (1:N).** Pulsa **New 1:N Relation**, haz clic primero en **ESTUDIO** (lado 1) y después en **VIDEOJUEGO** (lado N). En sus propiedades, pon el nombre `desarrolla` e indica la opcionalidad de cada extremo: un estudio puede no tener juegos (el 0 del `(0,N)` junto a VIDEOJUEGO en nuestro diagrama) y todo juego tiene estudio (el 1 del `(1,1)` junto a ESTUDIO).
2. **VIDEOJUEGO pertenece a GENERO (N:M).** Pulsa **New M:N Relation** y haz clic en las dos entidades.
3. **VIDEOJUEGO tiene LOGRO (débil).** Crea una relación 1:N de VIDEOJUEGO a LOGRO y, en sus propiedades, marca **Identifying**. Así la clave de VIDEOJUEGO formará parte de la clave de LOGRO, como en la regla 10.

Paso 4 · Transformar al modelo relacional

1. Pulsa el botón **Engineer to Relational Model** (el icono de la doble flecha `>>` en la barra de herramientas).
2. En el cuadro de diálogo revisa qué se va a transformar y pulsa **Engineer**.
3. Abre el diagrama relacional y comprueba el resultado:
   - VIDEOJUEGO tiene una columna nueva que es **clave ajena** hacia ESTUDIO, marcada como obligatoria.
   - Ha aparecido una **tabla intermedia** para la N:M entre VIDEOJUEGO y GENERO, con clave primaria compuesta.
   - La clave primaria de LOGRO está formada por la clave de VIDEOJUEGO **y** `num_logro`.

Renombra lo que genera la herramienta

Data Modeler pone nombres automáticos a las columnas de clave ajena y a la tabla intermedia (normalmente combinando los nombres de las tablas). Cámbialos en el modelo relacional para que coincidan con tu esquema: `id_estudio`, `JUEGO_GENERO`…

Paso 5 · Jerarquías (opcional)

Si quieres probar la especialización de VIDEOJUEGO, crea las entidades `DE_PAGO` y `FREE_TO_PLAY`, y en las propiedades de cada una indica que su supertipo (*Super Type*) es VIDEOJUEGO. En las propiedades del supertipo puedes elegir la estrategia de transformación (*Subtree Generation*), que corresponde a las tres del apartado 5.9:

Opción en Data Modeler	Estrategia del apartado 5.9
Table for each Entity	A. Una tabla por entidad
Table per Child	B. Solo subtipos
Single Table	C. Una sola tabla

Paso 6 · Generar el script DDL

1. Con el modelo relacional abierto, pulsa **Generate DDL** en la barra de herramientas (o **File → Export → DDL File**).
2. Elige como base de datos la versión de Oracle que usas en clase y pulsa **Generate**.
3. En el cuadro de opciones puedes dejar todo por defecto y pulsar **OK**. Aparece el editor con el script: sentencias `CREATE TABLE`, `ALTER TABLE ... ADD CONSTRAINT ... PRIMARY KEY` y `FOREIGN KEY`.
4. Guárdalo con **Save** como `pixelvault.sql`.

Deberías ver algo parecido a esto para LOGRO (los nombres y tipos exactos dependerán de lo que hayas configurado):

Fragmento del DDL generado (orientativo)

```
CREATE TABLE logro (
    id_juego     NUMBER NOT NULL,
    num_logro    NUMBER NOT NULL,
    nombre       VARCHAR2(100) NOT NULL,
    descripcion  VARCHAR2(400),
    puntos       NUMBER
);

ALTER TABLE logro ADD CONSTRAINT logro_pk PRIMARY KEY ( id_juego, num_logro );

ALTER TABLE logro
    ADD CONSTRAINT logro_videojuego_fk FOREIGN KEY ( id_juego )
        REFERENCES videojuego ( id_juego );
```

Comprueba el script con lo que sabes

- ¿La clave primaria de LOGRO tiene dos columnas? Debería (entidad débil por identificación).
- ¿La clave ajena de LOGRO tiene `ON DELETE CASCADE`? Por defecto **no**: tendrás que indicarlo en las propiedades de la clave ajena, como decidimos en el apartado 6.
- ¿Aparece alguna restricción `CHECK` para `pegi`? No, porque no se la hemos dicho: las restricciones del apartado 8 hay que añadirlas a mano.

Ejecutar este script en Oracle Database XE lo haremos en la **UD 5**.

9.4 Práctica: PixelVault completo en Data Modeler

Práctica 9 · Entregable

Modela en Data Modeler el caso PixelVault **completo** y entrega:

1. El diseño (`.dmd` y su carpeta) comprimido en un zip.
2. Una captura del **modelo lógico** y otra del **modelo relacional**.
3. El **script DDL** generado (`pixelvault.sql`).
4. Un documento breve donde compares las tablas generadas con el esquema del apartado 5.11 y expliques cualquier diferencia.

Recuerda las tres limitaciones: BIBLIOTECA y JUEGA serán entidades, y RESENA colgará de BIBLIOTECA.

Criterio de corrección	Peso
Todas las entidades y atributos del diccionario de datos, con sus identificadores	25 %
Relaciones con cardinalidad y opcionalidad correctas	25 %
Entidad débil (Identifying) y jerarquías configuradas	15 %
Modelo relacional coherente con el esquema del apartado 5 (nombres revisados)	20 %
Script DDL generado y explicación de las diferencias	15 %

Resumen

- **Data Modeler** es la herramienta gratuita de Oracle para diseñar bases de datos: modelo lógico, relacional y físico.
- Usa notación **Barker** o **pata de gallo**, no Chen: mismos conceptos, distinto dibujo.
- Solo admite relaciones **binarias sin atributos**: las N:M con atributos, las ternarias y las agregaciones se modelan como entidades.
- **Engineer to Relational Model** hace el paso a tablas; **Generate DDL** produce el script para Oracle.

- El script generado **no** incluye lo que no le hayas dicho: acciones de borrado, `CHECK`, restricciones no representables.

UD 2 · REPASO

Ejercicios resueltos

Doce ejercicios de dificultad creciente, todos del mundo de los videojuegos pero **fuera de PixelVault**, para que practiques con situaciones nuevas. Intenta resolver cada uno en papel antes de desplegar la solución.

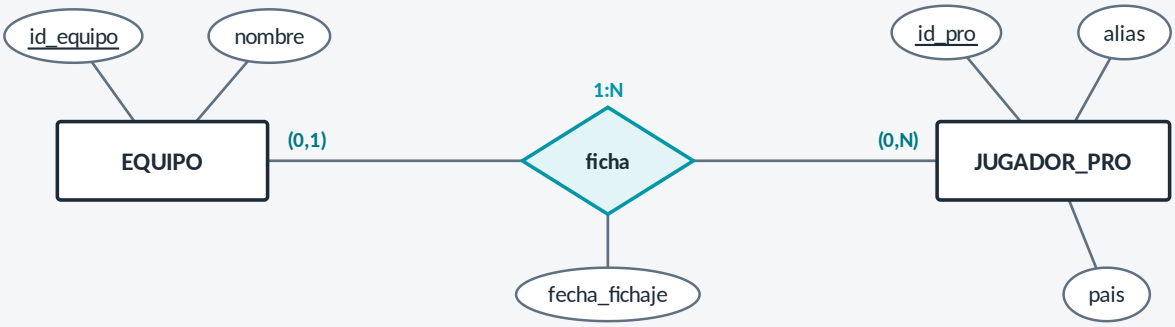
Bloque	Ejercicios	Apartados
Paso a tablas	1 a 8	5 y 6
Normalización	9 a 11	7
Restricciones no representables	12	8

Recuerda el convenio de cardinalidad del **extremo opuesto**: el par `(mín, máx)` junto a una entidad B indica cuántas ocurrencias de B se relacionan con **una** ocurrencia de la otra entidad.

Bloque 1 · Paso a tablas

Ejercicio 1 · 1:N con mínimo 0 en el lado 1

Una liga de eSports registra sus **equipos** y sus **jugadores profesionales**. Un jugador puede estar en un equipo o ser agente libre (sin equipo). Interesa la fecha en la que fichó por su equipo actual.



Obtén el esquema relacional e indica si la clave ajena admite nulos.

Solución

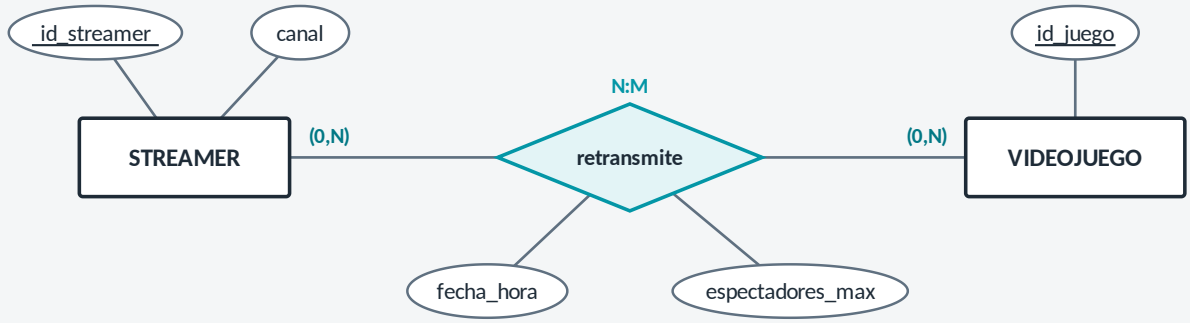
La relación es 1:N: junto a EQUIPO está el máximo 1 y junto a JUGADOR_PRO la N. Hay **propagación de clave** de EQUIPO (lado 1) a JUGADOR_PRO (lado N). El par junto a EQUIPO es `(0,1)`: un jugador puede no tener equipo, así que la clave ajena **admite nulos** (agentes libres). El atributo de la relación, `fecha_fichaje`, se propaga con ella.

```
EQUIPO(id_equipo, nombre)
JUGADOR_PRO(id_pro, alias, pais, id_equipo → EQUIPO NULL, fecha_fichaje NULL)
```

`fecha_fichaje` también será nula para los agentes libres. Sería razonable documentar la restricción «`fecha_fichaje` es nula si y solo si `id_equipo` es nulo», que se puede implementar con un `CHECK`.

Ejercicio 2 · N:M con combinaciones repetidas

Una plataforma de *streaming* registra qué **videojuegos** retransmite cada **streamer**. Un mismo streamer retransmite el mismo juego muchas veces, en días y horas distintos; de cada retransmisión interesa la fecha y hora de inicio y el número máximo de espectadores.



Solución

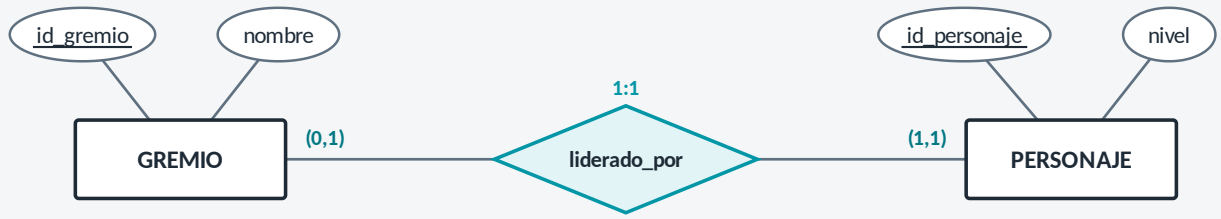
La N:M genera una tabla. Pero si la clave primaria fuera solo `(id_streamer, id_juego)`, el streamer no podría retransmitir el mismo juego dos veces. Hay que añadir `fecha_hora` a la clave:

```
STREAMER(id_streamer, canal)
VIDEOJUEGO(id_juego, ...)
RETRANSMISION(id_streamer → STREAMER, id_juego → VIDEOJUEGO, fecha_hora,
   espectadores_max)
```

Pregunta de ampliación: ¿haría falta `id_juego` en la clave? Un streamer no puede empezar dos retransmisiones a la vez, así que `(id_streamer, fecha_hora)` ya identifica la fila. Esa sería una clave primaria más ajustada, e `id_juego` quedaría como clave ajena `NOT NULL`. Las dos soluciones se aceptan si se justifican.

Ejercicio 3 · 1:1 con mínimos distintos

En un juego de rol online, cada **gremio** está liderado por exactamente un **personaje**, y un personaje puede liderar como mucho un gremio (la mayoría no lidera ninguno).



Solución

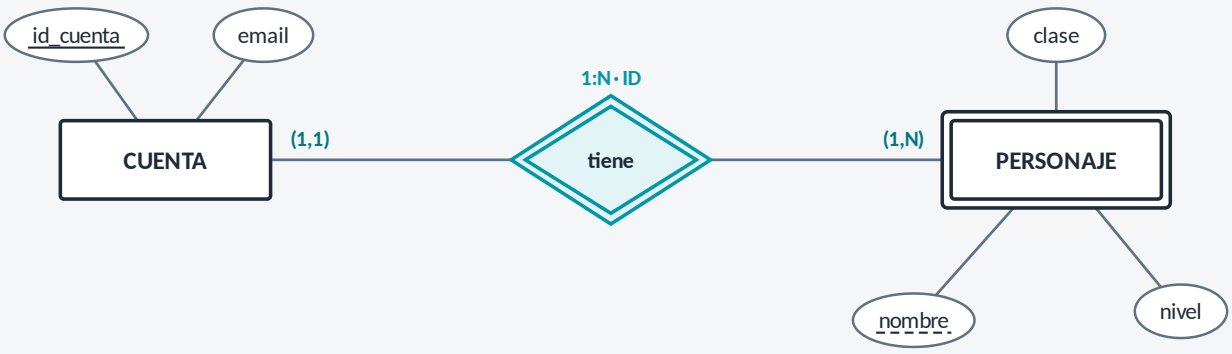
Es 1:1. Junto a PERSONAJE está `(1,1)`: todo gremio tiene exactamente un líder. Junto a GREMIO está `(0,1)`: un personaje lidera como mucho un gremio, y la mayoría ninguno. Según la regla 7, se propaga la clave de la entidad que tiene el `(1,1)` junto a ella, PERSONAJE, hacia la tabla GREMIO, con `NOT NULL` y `UNIQUE`:

```
PERSONAJE(id_personaje, nivel)
GREMIO(id_gremio, nombre, id_lider → PERSONAJE NOT NULL UNIQUE)
```

Si la hubiéramos puesto en PERSONAJE (`id_gremio_liderado`), la columna sería nula en casi todos los personajes. El `UNIQUE` impide que un personaje lidere dos gremios; sin él la relación sería 1:N.

Ejercicio 4 · Entidad débil por identificación

Cada **cuenta** de un juego online puede tener varios **personajes**. El nombre de un personaje solo es único **dentro de su cuenta** (dos cuentas distintas pueden tener un personaje llamado «Aragorn»). Si se borra la cuenta, se borran sus personajes. Toda cuenta tiene al menos un personaje.



Solución

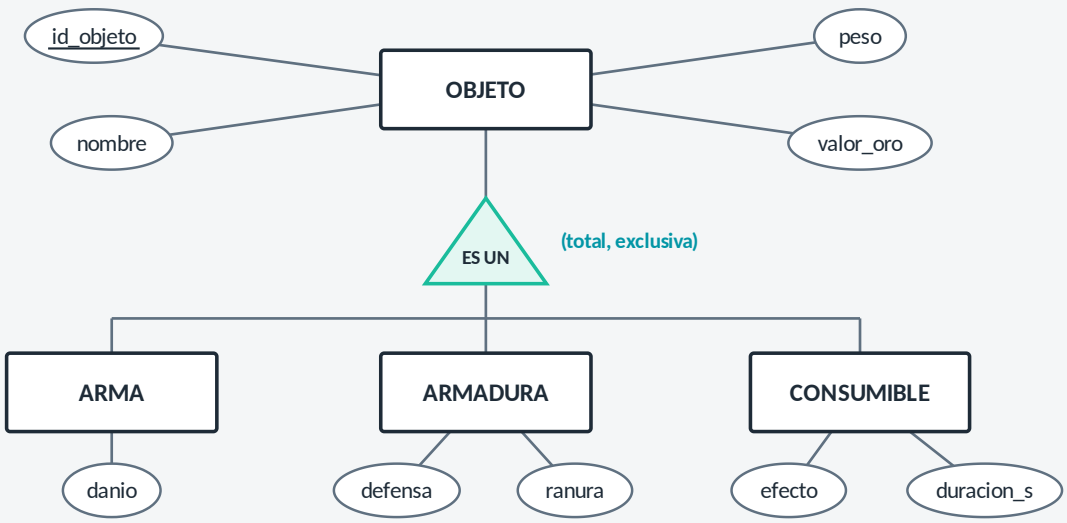
PERSONAJE es débil por identificación: la clave de CUENTA se **propaga** a PERSONAJE y, junto con el discriminador `nombre`, forma su clave primaria. La clave ajena va con borrado en cascada.

> **CUENTA**(**id_cuenta**, email UNIQUE)
> **PERSONAJE**(***id_cuenta → CUENTA***, **nombre**, clase, nivel) — *ON DELETE CASCADE*

El `(1,N)` junto a PERSONAJE («toda cuenta tiene al menos un personaje») **no** queda garantizado por las tablas: es una restricción no representable que hay que documentar.

Ejercicio 5 · Jerarquía: elegir estrategia

Los **objetos** de un juego tienen identificador, nombre, peso y valor en oro. Todo objeto es exactamente de uno de estos tipos: **arma** (daño), **armadura** (defensa y ranura) o **consumible** (efecto y duración en segundos). Los subtipos no participan en ninguna relación propia.



Transforma la jerarquía con las estrategias B y C, y di cuál elegirías.

Solución

Estrategia B (solo subtipos). Es posible porque la jerarquía es total y exclusiva:

```
ARMA(id_objeto, nombre, peso, valor_oro, danio)
ARMADURA(id_objeto, nombre, peso, valor_oro, defensa, ranura)
CONSUMIBLE(id_objeto, nombre, peso, valor_oro, efecto, duracion_s)
```

Problema: nada impide que el mismo `id_objeto` aparezca en dos tablas (habría que documentarlo), y si otra entidad (por ejemplo, el inventario) quisiera referenciar «un objeto», no tendría una única tabla a la que apuntar.

Estrategia C (una sola tabla):

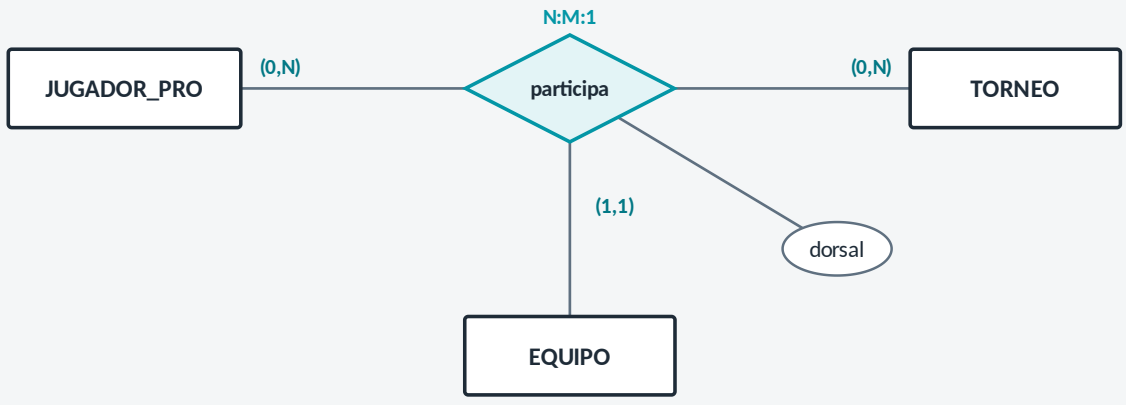
```
OBJETO(id_objeto, nombre, peso, valor_oro, tipo, danio NULL, defensa NULL, ranura
   NULL, efecto NULL, duracion_s NULL)
```

Con un `CHECK` que obligue a que `tipo IN ('ARMA','ARMADURA','CONSUMIBLE')` y a que solo estén rellenas las columnas de su tipo.

Elección: la **C**. Los subtipos tienen solo uno o dos atributos cada uno y ninguna relación propia, así que una sola tabla es más sencilla de consultar y cualquier otra tabla podrá referenciar a OBJETO. Los nulos son pocos y están controlados por el `CHECK`. (La estrategia A también sería correcta, aunque más costosa de consultar.)

Ejercicio 6 · Ternaria con un lado 1

En una liga, un **jugador profesional** participa en **torneos** con un **equipo**. En un mismo torneo, un jugador juega siempre con **un único equipo**, pero en torneos distintos puede ir con equipos distintos. Interesa el dorsal que lleva en cada torneo.



Solución

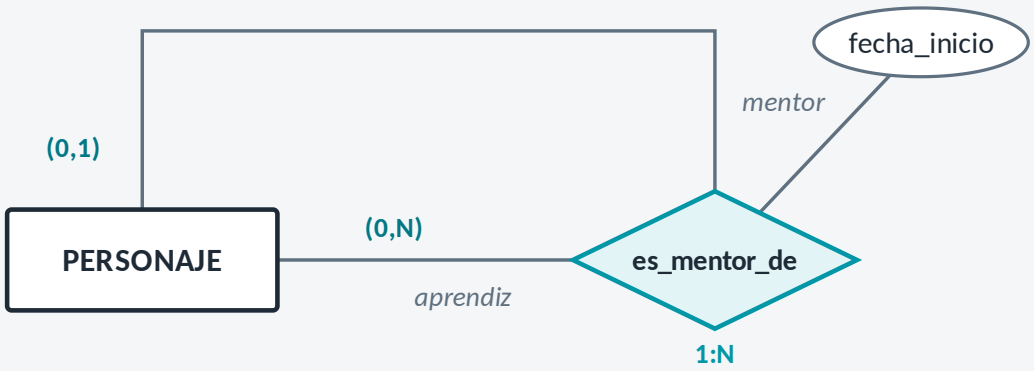
Fijando jugador y torneo, el equipo es **uno**: por eso EQUIPO tiene el 1 y la relación es N:M:1. La tabla lleva las tres claves ajenas, pero la clave primaria es solo **la de los otros dos**:

```
PARTICIPA(id_pro → JUGADOR_PRO, id_torneo → TORNEO, id_equipo → EQUIPO NOT NULL,
   dorsal)
```

Si pusiéramos las tres en la clave, la base de datos aceptaría que un jugador estuviera en el mismo torneo con dos equipos, que es justo lo que prohíbe el enunciado.

Ejercicio 7 · Reflexiva 1:N

Un personaje veterano puede ser **mentor** de varios personajes novatos; cada novato tiene como mucho un mentor. Interesa la fecha en la que empezó la mentoría.



Solución

Es una 1:N sobre la misma tabla: junto a la pata *mentor* está `(0,1)` (cada aprendiz tiene como mucho un mentor) y junto a la pata *aprendiz*, `(0,N)` (cada mentor, muchos aprendices). La clave se propaga del papel 1 (mentor) al papel N (aprendiz): cada personaje guarda la referencia a **su** mentor, que admite nulos porque el mínimo es 0. El atributo de la relación se propaga con ella.

```
PERSONAJE(id_personaje, ..., id_mentor → PERSONAJE NULL, fecha_inicio_mentoria NULL)
```

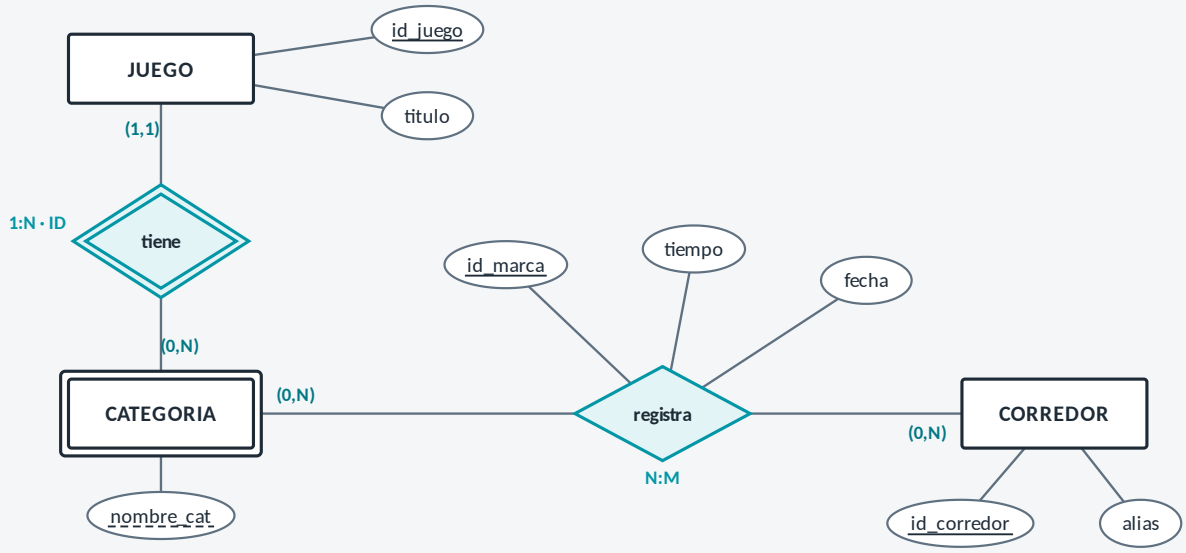
Restricciones a documentar: un personaje no puede ser su propio mentor (`CHECK (id_mentor <> id_personaje)`), y no debería haber ciclos (A mentor de B y B mentor de A), que ya no se puede comprobar con un `CHECK` .

Ejercicio 8 · Caso integrador: speedrun

Una web de *speedrun* registra marcas. De cada **juego** se guarda identificador y título. Cada juego tiene varias **categorías** («Any%», «100%», «Glitchless»…) cuyo nombre solo es único dentro del juego. De cada **corredor** se guarda identificador y alias. Un corredor registra muchas marcas en una categoría (puede mejorar su tiempo varias veces el mismo día); de cada marca se guarda un identificador, el tiempo y la fecha.

Diseña el diagrama E/R y obtén el esquema relacional.

Solución



CATEGORIA es débil por identificación (Any% de Hollow Knight no es Any% de Celeste). Como un corredor puede repetir marca en la misma categoría y el mismo día, ni `(corredor, categoría)` ni `(corredor, categoría, fecha)` identifican una marca: por eso el enunciado da un `id_marca`, que será la clave primaria.

```
JUEGO(id_juego, titulo)
CATEGORIA(id_juego → JUEGO, nombre_cat)  — ON DELETE CASCADE
CORREDOR(id_corredor, alias UNIQUE)
MARCA(id_marca, tiempo, fecha, id_corredor → CORREDOR NOT NULL, id_juego NOT NULL,
   nombre_cat NOT NULL)
(id_juego, nombre_cat) → CATEGORIA
```

Fíjate en que la clave ajena hacia CATEGORIA es **compuesta**: lleva las dos columnas de su clave primaria.

Bloque 2 · Normalización

Ejercicio 9 · Dependencias y forma normal

Dada la tabla

```
INSCRIPCION(id_torneo, nombre_torneo, fecha_torneo, id_equipo, nombre_equipo, capitan,
   email_capitan, cuota_pagada)
```

y las reglas: cada torneo tiene un nombre y una fecha; cada equipo tiene un nombre y un único capitán; cada capitán tiene un único correo; la cuota pagada depende del equipo y del torneo.

1. Escribe las dependencias funcionales.
2. Indica la clave primaria.
3. ¿En qué forma normal está? Normalízala hasta 3FN.

Solución

1. Dependencias funcionales

- `id_torneo → nombre_torneo, fecha_torneo`
- `id_equipo → nombre_equipo, capitan`
- `capitan → email_capitan`
- `(id_torneo, id_equipo) → cuota_pagada`

2. Clave primaria: `(id_torneo, id_equipo)`.

3. Forma normal: está en **1FN** (valores atómicos), pero **no en 2FN**: `nombre_torneo` y `fecha_torneo` dependen solo de `id_torneo`, y los datos del equipo solo de `id_equipo` (dependencias parciales).

2FN:

```
TORNEO(id_torneo, nombre_torneo, fecha_torneo)
EQUIPO(id_equipo, nombre_equipo, capitan, email_capitan)
INSCRIPCION(id_torneo → TORNEO, id_equipo → EQUIPO, cuota_pagada)
```

3FN: en EQUIPO hay una dependencia transitiva, `id_equipo → capitan → email_capitan`:

```
CAPITAN(capitan, email_capitan)
EQUIPO(id_equipo, nombre_equipo, capitan → CAPITAN)
```

TORNEO e INSCRIPCION ya estaban en 3FN.

Ejercicio 10 · Normalizar un acta de partida

El árbitro de un torneo rellena este acta por cada partida:

id_partida	fecha	id_torneo	nombre_torneo	sede	ciudad_sede	jugadores (id, alias, equipo, kills, muertes)
P-17	2026-10-03	T-2	Rioja Masters	S-1	Logroño	(7, Ruedas, Los Tempranillos, 21, 8); (9, Gárgola, Los Tempranillos, 15, 12); (12, Nube, Riojaforja, 11, 19)

Reglas: cada partida pertenece a un torneo; cada torneo se juega en una sede; cada sede está en una ciudad; cada jugador pertenece a un equipo; las kills y muertes son de un jugador en una partida. Normaliza hasta 3FN.

Solución

1FN. Se elimina el grupo repetitivo (una fila por jugador y partida). Clave: `(id_partida, id_jugador)`.

> **ACTA**(**id_partida**, **id_jugador**, fecha, id_torneo, nombre_torneo, sede, ciudad_sede, alias, equipo, kills, muertes)

Dependencias: `id_partida → fecha, id_torneo`; `id_torneo → nombre_torneo, sede`; `sede → ciudad_sede`; `id_jugador → alias, equipo`; `(id_partida, id_jugador) → kills, muertes`.

2FN. Se separan las dependencias parciales:

> **PARTIDA**(**id_partida**, fecha, id_torneo, nombre_torneo, sede, ciudad_sede)
> **JUGADOR**(**id_jugador**, alias, equipo)
> **ESTADISTICA**(***id_partida → PARTIDA***, ***id_jugador → JUGADOR***, kills, muertes)

3FN. En PARTIDA hay dos transitivas encadenadas: `id_partida → id_torneo → sede → ciudad_sede`.

> **SEDE**(**sede**, ciudad_sede)
> **TORNEO**(**id_torneo**, nombre_torneo, *sede → SEDE*)
> **PARTIDA**(**id_partida**, fecha, *id_torneo → TORNEO*)
> **JUGADOR**(**id_jugador**, alias, equipo)
> **ESTADISTICA**(***id_partida → PARTIDA***, ***id_jugador → JUGADOR***, kills, muertes)

Si del equipo interesaran más datos (patrocinador, entrenador…), `equipo` pasaría a ser clave ajena hacia una tabla EQUIPO. Con solo el nombre, se puede dejar como atributo.

Ejercicio 11 · Forma normal de Boyce-Codd

En un torneo en línea, cada **servidor** se asigna a un único torneo, y cada jugador juega cada torneo en un único servidor. Un torneo usa varios servidores.

```
ASIGNACION(id_jugador, id_torneo, servidor)
```

¿Está en 3FN? ¿Y en FNBC? Si no lo está, descomponla.

Solución

Dependencias: `(id_jugador, id_torneo) → servidor` y `servidor → id_torneo`.

Claves candidatas: `(id_jugador, id_torneo)` y `(id_jugador, servidor)`.

- **3FN: sí.** La única dependencia cuyo determinante no es clave es `servidor → id_torneo`, pero `id_torneo` forma parte de una clave candidata.
- **FNBC: no.** `servidor` es un determinante y no es clave candidata.

Descomposición:

```
SERVIDOR(servidor, id_torneo)
ASIGNACION(id_jugador, servidor → SERVIDOR)
```

Se pierde la dependencia `(id_jugador, id_torneo) → servidor`: nada impide ahora asignar a un jugador dos servidores del mismo torneo. Hay que documentarlo como restricción no representable (es el mismo caso que el de los entrenadores del apartado 7.6).

Bloque 3 · Restricciones no representables

Ejercicio 12 · Analizar y documentar

Para la liga de eSports de los ejercicios 1 y 6 se han obtenido estas tablas:

```
EQUIPO(id_equipo, nombre, max_jugadores)
JUGADOR_PRO(id_pro, alias, fecha_nacimiento, id_equipo → EQUIPO NULL, fecha_fichaje
  NULL)
TORNEO(id_torneo, nombre, fecha_inicio, fecha_fin, edad_minima)
PARTICIPA(id_pro → JUGADOR_PRO, id_torneo → TORNEO, id_equipo → EQUIPO NOT NULL,
  dorsal)
```

Y estas reglas del cliente:

1. Un torneo no puede terminar antes de empezar.
2. El dorsal va del 1 al 99.
3. Un equipo no puede tener más jugadores que su `max_jugadores`.

4. Para participar en un torneo hay que tener, el día de inicio, al menos la `edad_minima` del torneo.
5. En un torneo, dos jugadores del mismo equipo no pueden llevar el mismo dorsal.
6. Todo torneo tiene al menos dos equipos participantes.

Clasifica cada regla e indica cómo se implementaría.

Solución

Regla	Tipo	¿Qué mira?	Implementación
1	Entre columnas de la misma fila	TORNEO	`CHECK (fecha_fin >= fecha_inicio)`
2	De dominio	PARTICIPA	`CHECK (dorsal BETWEEN 1 AND 99)`
3	De agregado (cuenta filas)	JUGADOR_PRO, EQUIPO	Disparador (UD 6)
4	Entre tablas	PARTICIPA, JUGADOR_PRO, TORNEO	Disparador (UD 6)
5	De unicidad compuesta	PARTICIPA	`UNIQUE (id_torneo, id_equipo, dorsal)`
6	Mínimo en una relación	PARTICIPA, TORNEO	Procedimiento de alta o comprobación antes de cerrar la inscripción (UD 6)

La regla 5 es la trampa del ejercicio: parece no representable, pero se resuelve con una **clave alternativa compuesta**. Merece la pena comprobar siempre si una regla se puede expresar con `UNIQUE` o `CHECK` antes de mandarla a un disparador.

Ficha de la regla 4:

- **Código:** R-04 · **Nombre:** Edad mínima del torneo.
- **Descripción:** la edad del jugador, calculada a la fecha de inicio del torneo, debe ser mayor o igual que `edad_minima` .
- **Tablas:** PARTICIPA (al insertar o modificar), JUGADOR_PRO y TORNEO (se consultan).
- **Acción:** rechazar la inscripción.
- **Implementación prevista:** disparador `BEFORE INSERT OR UPDATE` sobre PARTICIPA.

UD 2 · REPASO

Autoevaluación

Treinta preguntas para comprobar lo que has aprendido. Responde cada una antes de desplegar la solución. Si fallas alguna, vuelve al apartado indicado.

Proceso de diseño y modelo E/R

1. ¿Cuál de estas fases del diseño depende del SGBD concreto que se vaya a usar?

- a) El análisis de requisitos
- b) El diseño conceptual
- c) El diseño lógico
- d) El diseño físico

Solución

d) El diseño físico decide tipos de dato, índices y almacenamiento de un gestor concreto (Oracle). El conceptual (E/R) no depende de nada, y el lógico solo del tipo de modelo (relacional). Repasa el apartado 1.2.

2. En una hoja de cálculo de ventas, al anular el único pedido de un estudio desaparece toda la información de ese estudio. ¿Qué tipo de anomalía es?

- a) De inserción
- b) De borrado
- c) De modificación
- d) De integridad referencial

Solución

b) Es una anomalía de borrado: al eliminar un dato se pierde otro que no se quería eliminar. Repasa el apartado 1.1.

3. ¿Cómo se representa en notación Chen un atributo que puede tomar varios valores para la misma ocurrencia, como los idiomas de un juego?

- a) Con una elipse discontinua
- b) Con una elipse doble
- c) Con el nombre subrayado
- d) Con un doble rectángulo

Solución

b) Elipse doble = multivaluado. La discontinua es para los derivados, el subrayado para los identificadores y el doble rectángulo para las entidades débiles. Repasa el apartado 2.3.

4. ¿Por qué no se debe guardar la edad de un jugador como atributo?

Solución

Porque es un **atributo derivado**: se calcula a partir de la fecha de nacimiento. Si se guardara, dejaría de ser correcta con el paso del tiempo y habría que actualizarla continuamente. Se guarda la fecha de nacimiento y la edad se calcula cuando se necesita. Repasa el apartado 2.3.

5. En la relación ESTUDIO `(1,1)` *desarrolla* VIDEOJUEGO `(0,N)`, ¿qué significa el `(1,1)` junto a ESTUDIO?

- a) Que cada estudio desarrolla exactamente un videojuego
- b) Que cada videojuego es desarrollado exactamente por un estudio
- c) Que hay exactamente un estudio en la base de datos
- d) Que la relación es 1:1

Solución

b) El par se escribe en el **extremo opuesto**: junto a ESTUDIO va cuántos estudios corresponden a **un** videojuego. Un videojuego lo desarrolla exactamente un estudio. La relación es 1:N (el máximo 1 está junto a ESTUDIO y la N junto a VIDEOJUEGO). Repasa el apartado 2.5.

6. Un dato como `fecha_adquisicion` (cuándo adquirió un jugador un juego concreto), ¿dónde debe colocarse en el diagrama?

Solución

Como **atributo de la relación** *posee*, porque no depende solo del jugador ni solo del juego, sino de la pareja. Si se pusiera en VIDEOJUEGO, solo se podría guardar una fecha por juego para todos los jugadores. Repasa el apartado 2.6.

7. ¿Por qué hay que etiquetar con roles los extremos de una relación reflexiva?

Solución

Porque las dos «patas» salen de la misma entidad y, sin roles, no se sabe cuál es cuál. En *es_secuela_de*, hay que distinguir qué juego es el *original* y cuál la *secuela*; si no, la cardinalidad es ambigua. Repasa el apartado 2.7.

8. ¿Se puede sustituir la ternaria JUGADOR *juega* VIDEOJUEGO en PLATAFORMA (con `horas_jugadas`) por tres relaciones binarias?

- a) Sí, siempre que las tres sean N:M
- b) Sí, pero hay que repetir `horas_jugadas` en las tres
- c) No, porque se perdería la información de qué jugador jugó a qué juego en qué plataforma
- d) No, porque el modelo E/R no admite relaciones binarias con atributos

Solución

c) Saber que el jugador tiene el juego, que usa la plataforma y que el juego está en la plataforma no permite deducir que haya jugado a ese juego en esa plataforma, ni cuántas horas. Repasa el apartado 2.8.

E/R ampliado

9. Los logros de PixelVault se numeran dentro de cada juego. ¿Qué tipo de entidad es LOGRO?

- a) Fuerte
- b) Débil por existencia
- c) Débil por identificación
- d) Subtipo de VIDEOJUEGO

Solución

c) El número de logro solo identifica dentro del juego: para identificar un logro hace falta el juego y el número. Repasa el apartado 3.1.

10. Un DLC tiene código propio pero no puede existir sin su juego. ¿Qué diferencia hay entre su clave primaria y la de LOGRO al pasar a tablas?

Solución

DLC es débil **por existencia**: su clave primaria es solo `id_dlc`, y `id_juego` es una clave ajena `NOT NULL`. LOGRO es débil **por identificación**: su clave primaria es `(id_juego, num_logro)`. En los dos casos la clave ajena suele llevar borrado en cascada. Repasa los apartados 3.1 y 5.8.

11. «Una misma persona puede ser jugador y desarrollador a la vez, y hay usuarios que no son ni una cosa ni otra.» ¿Qué tipo de jerarquía describe?

- a) Total y exclusiva
- b) Total y solapada
- c) Parcial y exclusiva
- d) Parcial y solapada

Solución

d) Parcial, porque hay usuarios sin subtipo; solapada, porque se puede pertenecer a los dos subtipos a la vez. Repasa el apartado 3.2.

12. ¿Qué ventaja tiene conectar la relación *incluye* al subtipo DE_PAGO en lugar de a VIDEOJUEGO?

Solución

Que el propio diagrama (y después la clave ajena hacia JUEGO_PAGO) impide incluir en un pedido un juego *free-to-play*. La regla de negocio queda recogida en la estructura y no hay que comprobarla aparte. Repasa el apartado 3.2.

13. ¿Para qué sirve la agregación en el caso de las reseñas?

Solución

Para relacionar la reseña con la **posesión** (la pareja jugador–juego de *posee*) y no con el jugador y el juego por separado. Así el diagrama expresa que solo se reseña un juego que se tiene y, con el `(0,1)` junto a RESEÑA, que hay una reseña como máximo por posesión. Repasa el apartado 3.3.

Modelo relacional y paso a tablas

14. En el modelo relacional, ¿qué es una tupla?

- a) Una columna
- b) Una fila
- c) Una tabla
- d) Un conjunto de valores válidos

Solución

b) Tupla = fila. Relación = tabla, atributo = columna, dominio = conjunto de valores válidos. Repasa el apartado 4.2.

15. ¿Cuál de estas afirmaciones sobre NULL es correcta?

- a) NULL es lo mismo que 0 en las columnas numéricas
- b) Una clave primaria puede ser NULL si es compuesta
- c) NULL indica ausencia de valor: desconocido o no aplicable
- d) `NULL = NULL` es verdadero

Solución

c) NULL no es 0 ni cadena vacía, ninguna parte de una clave primaria puede ser nula, y comparar NULL con NULL da *desconocido*. Repasa el apartado 4.5.

16. ¿Qué diferencia hay entre clave candidata y clave alternativa?

Solución

Las claves candidatas son todos los conjuntos mínimos de atributos que identifican cada fila. De ellas se elige una como **primaria**; las demás son **alternativas** (se declaran `UNIQUE`). En JUGADOR, `id_usuario` y `nick` son candidatas; la primaria es `id_usuario` y `nick` es alternativa. Repasa el apartado 4.4.

17. En una relación 1:N, ¿en qué tabla se coloca la clave ajena?

- a) En la tabla de la entidad del lado 1
- b) En la tabla de la entidad del lado N: la clave del lado 1 se propaga hacia ella
- c) En una tabla nueva
- d) En las dos tablas

Solución

b) Es la **propagación de clave**: cada fila del lado N apunta a una sola fila del lado 1. En *desarrolla*, la clave de ESTUDIO se propaga a VIDEOJUEGO, y cada juego guarda el `id_estudio` de su estudio. Repasa el apartado 5.3.

18. Transforma el atributo multivaluado `{telefono}` de una entidad TIENDA(cod_tienda, nombre).

Solución

> **TIENDA**(**cod_tienda**, nombre)
> **TELEFONO_TIENDA**(***cod_tienda → TIENDA***, **telefono**)

La clave primaria de la tabla nueva es la combinación de la clave de la entidad y el valor. Repasa el apartado 5.2.

19. ¿Qué restricción convierte una clave ajena en la implementación de una relación 1:1 y no 1:N?

- a) NOT NULL
- b) UNIQUE
- c) CHECK
- d) ON DELETE CASCADE

Solución

b) Con `UNIQUE`, dos filas no pueden apuntar a la misma fila referenciada: cada una se relaciona con una como máximo en los dos sentidos. Repasa el apartado 5.5.

20. La ternaria *participa* entre JUGADOR_PRO, TORNEO y EQUIPO es N:M:1 (en un torneo, cada jugador va con un solo equipo). ¿Cuál es la clave primaria de la tabla resultante?

Solución

`(id_pro, id_torneo)`. La entidad del lado 1 (EQUIPO) no entra en la clave: queda como clave ajena `NOT NULL`. Si entrara, se permitiría que un jugador jugara un torneo con dos equipos. Repasa el ejercicio 6.

21. La jerarquía OBJETO → ARMA, ARMADURA, CONSUMIBLE es total y exclusiva y los subtipos no tienen relaciones propias. ¿Qué estrategia **no** sería adecuada si otra tabla (INVENTARIO) tuviera que referenciar a cualquier objeto?

- a) Una tabla por entidad
- b) Solo subtipos
- c) Una sola tabla
- d) Todas serían adecuadas

Solución

b) Con solo subtipos no existe una tabla OBJETO a la que apuntar: INVENTARIO necesitaría tres claves ajenas distintas. Repasa el apartado 5.9 y el ejercicio 5.

22. En DESBLOQUEA, ¿por qué la clave ajena hacia LOGRO tiene dos columnas?

Solución

Porque la clave primaria de LOGRO es compuesta, `(id_juego, num_logro)`, y una clave ajena debe llevar **todas** las columnas de la clave a la que apunta. Solo con `num_logro` no se sabría de qué juego es el logro. Repasa el apartado 5.8.

Integridad

23. ¿Qué dice la regla de integridad referencial?

Solución

Que cada valor de una clave ajena debe coincidir con un valor existente de la clave primaria referenciada, o ser NULL si la columna lo admite. No puede haber referencias a filas que no existen. Repasa el apartado 6.3.

24. ¿Cuál de estas acciones referenciales **no** existe en Oracle?

- a) Restringir el borrado (comportamiento por defecto)
- b) ON DELETE CASCADE
- c) ON DELETE SET NULL
- d) ON UPDATE CASCADE

> ✓ **Solución**
>
> **d)** Oracle no permite propagar modificaciones de la clave primaria. Es una razón más para usar claves que no cambien nunca. Repasa el apartado 6.4.

25. ¿Qué acción de borrado es la más adecuada para la clave ajena de LOGRO hacia VIDEOJUEGO? ¿Y para la de VIDEOJUEGO hacia ESTUDIO? Justifícalo.

> ✓ **Solución**
>
> LOGRO → VIDEOJUEGO: **CASCADE**, porque LOGRO es una entidad débil y un logro no tiene sentido sin su juego. VIDEOJUEGO → ESTUDIO: **restringir**, porque no debe poder borrarse un estudio del que se venden juegos; perderíamos quién los desarrolló. Repasa el apartado 6.4.

Normalización

26. Una tabla con clave primaria **simple** está en 1FN. ¿Qué se puede asegurar?

- a) Que está en 3FN
- b) Que está en 2FN
- c) Que está en FNBC
- d) Nada más

> ✓ **Solución**
>
> **b)** Si la clave tiene una sola columna, no puede haber dependencias de una *parte* de la clave, así que no hay dependencias parciales. Sí podría haber transitivas, así que no se puede asegurar la 3FN. Repasa el apartado 7.4.

27. En PEDIDO(num_pedido, fecha, nick, email) se cumple `nick → email`. ¿Qué forma normal incumple y cómo se arregla?

Solución

Incumple la **3FN**: `email` depende transitivamente de la clave a través de `nick`, que no es clave. Se arregla sacando la dependencia a otra tabla:

```
JUGADOR(nick, email)
PEDIDO(num_pedido, fecha, nick → JUGADOR)
```

Repasa el apartado 7.5.

28. ¿Qué exige la forma normal de Boyce-Codd que no exija la 3FN?

Solución

Que **todo determinante sea clave candidata**, sin excepciones. La 3FN permite una dependencia X → A con X no clave si A forma parte de alguna clave candidata; la FNBC no lo permite. Repasa el apartado 7.6.

Restricciones no representables

29. ¿Cuál de estas reglas se puede implementar con un `CHECK`?

- a) Para reseñar hay que haber jugado al menos 2 horas
- b) Una secuela no puede salir antes que el juego original
- c) Un jugador no puede ser amigo de sí mismo
- d) Todo pedido tiene al menos una línea

Solución

c) Solo compara dos columnas de la **misma fila** de AMISTAD: `CHECK (id_jugador <> id_amigo)`. Las demás necesitan consultar otras filas u otras tablas (disparadores o procedimientos). Repasa el apartado 8.3.

30. En PixelVault, en *pertenece* aparece `(1,N)` junto a GÉNERO (todo juego tiene al menos un género). ¿Garantizan las tablas VIDEOJUEGO y JUEGO_GENERO esa regla? ¿Qué hay que hacer?

Solución

No. La clave ajena de JUEGO_GENERO garantiza que cada fila apunta a un juego que existe, pero nada obliga a que cada juego tenga filas en JUEGO_GENERO. El mínimo 1 de una N:M se pierde al pasar a tablas. Hay que **documentarlo** como restricción no representable (R-12) e implementarlo, por ejemplo, con un procedimiento de alta de juegos. Repasa el apartado 8.2.

UD 2 · REPASO

Glosario de la UD 2

Los términos clave de la unidad, por orden alfabético.

Acción referencial

> Lo que hace el SGBD con las filas que referencian a una fila que se borra: restringir el borrado, borrar en cascada (`ON DELETE CASCADE`) o poner la clave ajena a nulo (`ON DELETE SET NULL`).

Agregación

> Construcción del modelo E/R ampliado que trata una relación, junto con sus entidades, como una entidad de nivel superior para poder relacionarla con otras. Se dibuja con un recuadro.

Anomalía

> Problema que aparece en una tabla mal diseñada al insertar (no se puede guardar un dato sin otro), modificar (hay que cambiar el mismo dato en muchas filas) o borrar (se pierde información que no se quería borrar).

Atributo

> Propiedad de una entidad o de una relación. En el modelo relacional, cada columna de una tabla.

Atributo compuesto

> Atributo formado por otros atributos, como la dirección (calle, número, código postal…). Al pasar a tablas se sustituye por sus componentes.

Atributo derivado

> Atributo cuyo valor se calcula a partir de otros datos, como la edad o la nota media. No se guarda en las tablas.

Atributo multivaluado

> Atributo que puede tener varios valores para la misma ocurrencia, como los idiomas de un juego. Se dibuja con elipse doble y genera una tabla propia.

Cardinalidad de una relación

> En el modelo E/R, los pares (mín, máx) que se escriben en cada extremo de una relación. En estos apuntes se usa el convenio del **extremo opuesto**: junto a la entidad B se indica cuántas ocurrencias de B se relacionan con una ocurrencia de A. Los máximos determinan el tipo: 1:1, 1:N o N:M.

Cardinalidad de una tabla

> En el modelo relacional, número de filas que tiene una tabla en un momento dado.

Clave ajena (FK)

Columna o conjunto de columnas cuyos valores deben coincidir con los de la clave primaria (o alternativa) de otra tabla o de la misma. Implementa las relaciones.

Clave alternativa (UK)

Clave candidata que no se ha elegido como primaria. Sus valores no pueden repetirse; en Oracle se declara con `UNIQUE`.

Clave candidata

Conjunto mínimo de atributos que identifica cada fila de una tabla.

Clave primaria (PK)

Clave candidata elegida para identificar las filas. No puede repetirse ni contener nulos. Se subraya.

Clave subrogada

Identificador artificial, sin significado para el usuario (como `id_jugador`), que se usa como clave primaria porque no cambia nunca.

Dependencia funcional

Relación entre atributos X → Y que se cumple cuando cada valor de X determina un único valor de Y.

Dependencia parcial

Dependencia de un atributo respecto a una parte de una clave compuesta. Su ausencia define la 2FN.

Dependencia transitiva

Dependencia de un atributo respecto a la clave a través de otro atributo que no es clave (X → Z → Y). Su ausencia define la 3FN.

Desnormalización

Decisión consciente de dejar redundancia en una tabla normalizada para mejorar el rendimiento.

Diccionario de datos

Documento que acompaña al diagrama E/R con la lista completa de entidades, relaciones y atributos.

Discriminador

Atributo de una entidad débil por identificación que, junto con la clave de la entidad fuerte, identifica cada ocurrencia. También se llama clave parcial.

Dominio

Conjunto de valores válidos que puede tomar un atributo.

Entidad

Objeto, persona, concepto o suceso del mundo real del que interesa guardar información y que se puede distinguir de los demás. Se dibuja con un rectángulo.

Entidad débil

Entidad que depende de otra (la fuerte). Puede depender en existencia (no existe sin la fuerte) o en identificación (además, necesita la clave de la fuerte para identificarse).

Esquema relacional

Conjunto de tablas de una base de datos con sus columnas, claves primarias y claves ajenas.

Forma normal

Conjunto de condiciones que cumple una tabla. Las principales son 1FN (valores atómicos), 2FN (sin dependencias parciales), 3FN (sin dependencias transitivas) y FNBC (todo determinante es clave candidata).

Generalización

Proceso de reunir en un supertipo los atributos y relaciones comunes de varias entidades. El proceso inverso es la especialización.

Grado de una relación

En el modelo E/R, número de entidades que participan en una relación (reflexiva, binaria, ternaria). En el modelo relacional, número de columnas de una tabla.

Integridad de entidad

Regla que prohíbe que la clave primaria sea nula o se repita.

Integridad referencial

Regla que obliga a que cada valor de una clave ajena exista como clave primaria en la tabla referenciada, o sea nulo.

Jerarquía

Relación ES UN entre un supertipo y sus subtipos. Puede ser total o parcial, y exclusiva o solapada.

Modelo entidad/relación (E/R)

Modelo conceptual, propuesto por Peter Chen en 1976, que describe la realidad mediante entidades, atributos y relaciones.

Modelo relacional

Modelo lógico, propuesto por Edgar F. Codd en 1970, que organiza los datos en tablas (relaciones) enlazadas mediante claves.

Normalización

: Proceso de descomponer tablas para que cumplan las formas normales y eliminar así la redundancia y las anomalías.

NULL

: Marcador que indica ausencia de valor, porque es desconocido o no aplicable. No equivale a cero ni a cadena vacía.

Oracle SQL Data Modeler

: Herramienta gratuita de Oracle para diseñar modelos lógicos y relacionales y generar el script DDL.

Participación

: Indica si la relación es obligatoria (mínimo 1) u opcional (mínimo 0). Con el convenio del extremo opuesto, el mínimo escrito junto a B dice si toda ocurrencia de A tiene que estar relacionada con alguna B.

Propagación de clave

: Forma de transformar una relación 1:N (o 1:1) sin crear tabla: la clave primaria de la entidad del lado 1 se copia como clave ajena en la tabla del lado N. También se usa para formar la clave de las entidades débiles.

Relación (modelo E/R)

: Asociación entre entidades. Se dibuja con un rombo.

Relación (modelo relacional)

: Tabla. Conjunto de tuplas con los mismos atributos.

Relación reflexiva

: Relación de una entidad consigo misma, como *es_amigo_de*. Necesita roles en cada extremo.

Relación ternaria

: Relación en la que participan tres entidades a la vez.

Restricción no representable

: Regla de negocio que no se puede expresar en el diagrama E/R ni con las claves del esquema relacional, y que hay que documentar aparte.

Tupla

: Fila de una tabla.